

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

Бизнес-тезис «PlechiKi»: разработка и интеграция модели машинного обучения в веб-приложение для анализа и управления гардеробом

Обучающийся: Касьяненко Вера Михайловна

Квалификация: Бакалавриат

Факультет/институт/кластер: Факультет программной инженерии и компьютерной техники

Направление подготовки: 09.03.04 Программная инженерия

Образовательная программа: Нейротехнологии и программирование

Группа: P3420

Руководитель ВКР: Болдырева Елена Александровна

Обучающийся

Касьяненко Вера Михайловна

Дата: 18.05.2026

ЗАДАНИЕ

Основные вопросы, подлежащие разработке:

Техническое задание

Разработать модуль машинного обучения для веб-приложения анализа и управления гардеробом, обеспечивающий автоматическую классификацию атрибутов одежды, оценку совместимости элементов гардероба и формирование образов на основе пользовательского гардероба. Обеспечить интеграцию разработанного модуля в веб-приложение и взаимодействие через сервис обработки запросов.

Исходные данные

- Наборы изображений одежды для задач классификации атрибутов.
- Данные о сочетаниях элементов гардероба для обучения модели совместимости.
- Методы и алгоритмы машинного обучения для задач классификации изображений.
- Подходы к обработке и подготовке данных для обучения моделей машинного обучения.

Цель

Повышение релевантности рекомендаций по подбору одежды посредством разработки и интеграции модуля машинного обучения, решающего задачи классификации атрибутов и оценки визуальной совместимости элементов гардероба.

Задачи

- Провести исследование предметной области и анализ существующих решений в области цифрового управления гардеробом.
- Выполнить сбор и подготовку наборов данных для задач классификации атрибутов одежды и оценки совместимости элементов гардероба.
- Провести предобработку данных, включая очистку, преобразование признаков и формирование обучающей и тестовой выборок.
- Провести экспериментальное исследование моделей машинного обучения для задачи многоклассовой классификации атрибутов одежды.
- Провести экспериментальное исследование моделей для задачи оценки совместимости элементов гардероба.
- Выполнить сравнительный анализ алгоритмов и выбрать оптимальные модели на основе

показателей качества.

- Разработать алгоритм формирования рекомендаций образов на основе результатов классификации и оценки совместимости элементов гардероба.
- Интегрировать разработанный модуль машинного обучения в веб-приложение.

Перечень подлежащих разработке вопросов

- Анализ существующей проблемы управления персональным гардеробом.
- Исследование методов машинного обучения, применяемых для классификации изображений одежды.
- Исследование алгоритмов оценки совместимости элементов гардероба.
- Разработка архитектуры модуля машинного обучения для анализа гардероба.
- Разработка сервиса обработки запросов для взаимодействия веб-приложения с модулем машинного обучения.
- Интеграция разработанного модуля машинного обучения в веб-приложение.

Рекомендуемые материалы и пособия

- Ian Goodfellow, Yoshua Bengio, Aaron Courville - Deep Learning
- Christopher Bishop - Pattern Recognition and Machine Learning
- Richard Szeliski - Computer Vision: Algorithms and Applications
- Jason Brownlee - Deep Learning for Computer Vision
- Официальная документация Python, NumPy, Pandas, Scikit-learn, PyTorch
- Официальная документация FastAPI.

Содержание и предполагаемая структура работы

Список сокращений и условных обозначений

Термины и определения

Введение

1. Теоретические основы анализа гардероба с использованием методов машинного обучения
 - 1.1 Анализ проблемы управления персональным гардеробом
 - 1.2 Анализ цифровых сервисов управления гардеробом
 - 1.3 Задачи распознавания изображений одежды
 - 1.4 Методы машинного обучения для классификации изображений
 - 1.5 Подходы к оценке совместимости элементов гардероба
 - 1.6 Метрики оценки качества моделей
 - 1.7 Используемые инструменты и библиотеки
 - 1.8 Выводы по первой главе

- 2. Анализ и подготовка данных
 - 2.1 Источники и характеристика используемых данных
 - 2.2 Формирование набора данных изображений одежды
 - 2.3 Предобработка изображений и подготовка обучающих выборок
 - 2.4 Анализ распределения классов и характеристик данных
 - 2.5 Разделение данных на обучающую и тестовую выборки
 - 2.6 Выводы по второй главе
- 3. Разработка и сравнительная оценка моделей
 - 3.1 Разработка модели классификации атрибутов одежды
 - 3.2 Разработка модели оценки совместимости элементов гардероба
 - 3.3 Обучение моделей и подбор гиперпараметров
 - 3.4 Сравнительный анализ моделей по выбранным метрикам
 - 3.5 Разработка алгоритма формирования образов
 - 3.6 Выводы по третьей главе
- 4. Интеграция модуля машинного обучения в веб-приложение
 - 4.1 Архитектура системы
 - 4.2 Разработка сервиса обработки запросов
 - 4.3 Интеграция модели машинного обучения в веб-приложение
 - 4.4 Тестирование работы системы
 - 4.5 Выводы по четвертой главе
- Заключение
- Список использованных источников
- Приложение

Форма представления материалов ВКР:

Текст ВКР, Инвестиционная презентация

Дата выдачи задания

13.04.2026

Согласовано:

Руководитель ВКР Болдырева Елена Александровна, ФПИ и КТ, доцент (квалификационная категория "ординарный доцент"), осн.

АННОТАЦИЯ

Цель исследования

Повышение релевантности рекомендаций по подбору одежды посредством разработки и интеграции модуля машинного обучения, решающего задачи классификации атрибутов и оценки визуальной совместимости элементов гардероба.

Задачи, решаемые в ВКР

- Провести исследование предметной области и анализ существующих решений в области цифрового управления гардеробом.
- Выполнить сбор и подготовку наборов данных для задач классификации атрибутов одежды и оценки совместимости элементов гардероба.
- Провести предобработку данных, включая очистку, преобразование признаков и формирование обучающей и тестовой выборок.
- Провести экспериментальное исследование моделей машинного обучения для задачи многоклассовой классификации атрибутов одежды.
- Провести экспериментальное исследование моделей для задачи оценки совместимости элементов гардероба.
- Выполнить сравнительный анализ алгоритмов и выбрать оптимальные модели на основе показателей качества.
- Разработать алгоритм формирования рекомендаций образов на основе результатов классификации и оценки совместимости элементов гардероба.
- Интегрировать разработанный модуль машинного обучения в веб-приложение.

Краткая характеристика полученных результатов

Была разработана серверная часть модуля машинного обучения для веб-приложения "Plechiki", которое помогает вести личный гардероб пользователя и подбирать одежду на каждый день. Сервис работает с изображениями вещей напрямую, без ручного ввода описаний. Модуль обрабатывает изображения, классифицирует атрибуты вещей и строит векторные представления для каждого элемента гардероба. На этой основе оценивается совместимость вещей и формируются готовые комплекты одежды. Предварительно для обучения были найдены и обработаны датасеты изображений одежды с метками атрибутов и совместимостью вещей. По каждой прикладной задаче сравнивалось несколько архитектур, финальные модели были отобраны по точности классификации. ML-сервис оформлен как отдельный программный компонент, встроен в веб-приложение и проверен тестированием, охватившим отдельные функции модуля и сквозные

пользовательские сценарии.

Согласовано:

Руководитель ВКР Болдырева Елена Александровна, ФПИ и КТ, доцент (квалификационная категория "ординарный доцент"), осн.

14.05.2026

СОДЕРЖАНИЕ

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ.....	3
ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ.....	8
ВВЕДЕНИЕ.....	12
1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ АНАЛИЗА ГАРДЕРОБА С ИСПОЛЬЗОВАНИЕМ МЕТОДОВ МАШИННОГО ОБУЧЕНИЯ.....	14
1.1 Анализ проблемы управления персональным гардеробом	14
1.2 Анализ цифровых сервисов управления гардеробом.....	15
1.3 Методы машинного обучения для распознавания элементов одежды .	17
1.4 Подходы к оценке совместимости элементов гардероба.....	20
1.5 Метрики оценки качества моделей	22
1.6 Используемые инструменты и библиотеки.....	23
1.7 Выводы по первой главе.....	24
2 АНАЛИЗ И ПОДГОТОВКА ДАННЫХ	27
2.1 Источники и характеристика используемых данных.....	27
2.2 Формирование набора данных изображений одежды.....	29
2.3 Предобработка изображений и подготовка обучающих выборок	30
2.4 Анализ распределения классов и характеристик данных.....	34
2.5 Разделение данных на обучающую и тестовую выборки.....	40
2.6 Выводы по второй главе.....	42
3 РАЗРАБОТКА И СРАВНИТЕЛЬНАЯ ОЦЕНКА МОДЕЛЕЙ	44
3.1 Разработка модели классификации атрибутов одежды	44
3.2 Разработка модели оценки совместимости элементов гардероба	46
3.3 Обучение моделей и подбор гиперпараметров.....	48
3.4 Сравнительный анализ моделей по выбранным метрикам	51

3.5 Разработка алгоритма формирования образов.....	57
3.6 Выводы по третьей главе.....	61
4 ИНТЕГРАЦИЯ МОДУЛЯ МАШИННОГО ОБУЧЕНИЯ В ВЕБ-ПРИЛОЖЕНИЕ	63
4.1 Архитектура системы	63
4.2 Разработка сервиса обработки запросов.....	64
4.3 Интеграция модели машинного обучения в веб-приложение.....	68
4.4 Тестирование работы системы	71
4.5 Выводы по четвертой главе	75
ЗАКЛЮЧЕНИЕ	77
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	79
СПИСОК ИЛЛЮСТРАТИВНОГО МАТЕРИАЛА	86

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ

AdamW – адаптивный стохастический метод настройки весов нейросети

Albumentations – библиотека Python для аугментации изображений

AlexNet – архитектура сверточной нейронной сети

API (Application Programming Interface) – договоренность о вызовах между частями системы по сети

BEiT (BERT Pre-Training of Image Transformers) – метод самообучения визуальных трансформеров на восстановлении замаскированных токенов изображения

CatBoost – библиотека градиентного бустинга по деревьям

CLIP (Contrastive Language-Image Pre-Training) – мультимодальная модель, обучающая кодировщики на парах изображение-текст

CNN (Convolutional Neural Network) – нейросеть на сверточных слоях

ConvNeXt – сверточная архитектура с приемами трансформеров

CORS (Cross-Origin Resource Sharing) – правила междоменного доступа браузера к API через HTTP-заголовки

CPU (Central Processing Unit) – центральный процессор компьютера

DETR (DEtection TRansformer) – трансформерный детектор объектов, не требующий якорей и операции не максимального подавления

DINO (Distillation with No Labels) – метод самоконтролируемого обучения без размеченных данных

Docker – упаковка приложения с зависимостями в образ для переносимого запуска на другой машине

EfficientNet – CNN, в которой глубину сети, ширину каналов и разрешение входа меняют одним общим коэффициентом масштаба

EXIF (Exchangeable Image File Format) – служебный блок в файле снимка

F1-мера – гармоническое среднее точности и полноты

FastAPI – библиотека Python для построения HTTP API

FITB (fill-in-the-blank accuracy) – доля задач, где модель верно дополняет образ выбором из списка альтернатив

Git – распределенная система контроля версий исходного кода

GoogLeNet – глубокая CNN

GPU (Graphics Processing Unit) – вычислитель на видеокарте с массовым параллелизмом

GradientBoosting – реализация градиентного бустинга на решающих деревьях

HOG (Histogram of Oriented Gradients) – дескриптор изображения на основе гистограмм ориентаций градиентов

HTTP (HyperText Transfer Protocol) – протокол для обмена в Web по схеме запрос-ответ

ImageNet – корпус размеченных фото

Inception – модуль CNN, выполняющий свертки нескольких масштабов параллельно и объединяющий результаты конкатенацией

JSON (JavaScript Object Notation) – компактный текстовый формат вложенных записей и списков

k-NN (k-Nearest Neighbors) – алгоритм классификации, относящий объект к классу большинства из k ближайших соседей

Kubernetes – менеджер контейнерного кластера, который автоматизирует выкладку версий, число реплик сервиса и перезапуск при сбоях

LayerNorm (Layer Normalization) – нормализация активаций по признаковому вектору одного примера, а не по батчу

LightGBM – градиентный бустинг по деревьям с упором на скорость и табличные признаки

LSTM (Long Short-Term Memory) – рекуррентная нейросеть с механизмом памяти

MAE (Masked Autoencoder) – метод самообучения для восстановления случайно скрытых частей изображения

mAP (mean Average Precision) – усреднение по классам интегральной точности при многозначной классификации атрибутов

ML (Machine Learning) – машинное обучение, построение моделей, находящих закономерности в данных без явного программирования правил

MLP (Multi-Layer Perceptron) – многослойный перцептрон как классификатор над вектором признаков сверточного извлекавателя

MobileNet – компактные CNN на глубокой и точечной свертках

MoCo (Momentum Contrast) – метод контрастивного самообучения с очередью негативных примеров и экспоненциально усредняемым кодировщиком

NDCG (Normalized Discounted Cumulative Gain) – метрика ранжирования, штрафующая за низкие позиции релевантных объектов в списке выдачи

NGNN (Node-wise Graph Neural Network) – графовая нейросеть, обновляющая представление вершин с учетом соседей

NLP (Natural Language Processing) – область искусственного интеллекта, занимающаяся обработкой и генерацией текста на естественном языке

NumPy – библиотека Python для числовых вычислений

ONNX (Open Neural Network Exchange) – сериализованное описание графа модели для переноса весов между средами обучения

OpenAPI – машиночитаемый контракт HTTP-сервиса

pandas – библиотека Python для обработки и анализа табличных данных

Pillow – библиотека Python для чтения и преобразования растровых изображений

Polyvore – датасет комплектов одежды для обучения и оценки моделей совместимости

PR-AUC (Precision-Recall Area Under Curve) – площадь под кривой точность-полнота

Pydantic – библиотека Python для декларативной валидации схем данных запросов и ответов

pytest – среда автоматизированного тестирования Python-компонентов

PyTorch – библиотека тензорных вычислений и обучения нейросетей

R-CNN (Region-based Convolutional Neural Network) – семейство детекторов объектов с предварительным выделением регионов-кандидатов

RegNet – семейство CNN, полученных регулярным поиском по пространству архитектур

rembg – библиотека выделения объекта на фото на основе CNN

ResNet (Residual Network) – CNN с остаточными skip-связями между блоками

REST (Representational State Transfer) – соглашения о доступе к ресурсам по HTTP

ROC (Receiver Operating Characteristic) – кривая зависимости доли истинно положительных от доли ложно положительных при переборе порога классификатора

ROC-AUC – площадь под ROC-кривой как метрика бинарного классификатора при переборе порога

scikit-learn (sklearn) – библиотека классических алгоритмов ML, метрик и предобработки на Python

SIFT (Scale-Invariant Feature Transform) – алгоритм обнаружения ключевых точек, устойчивый к масштабу и повороту изображения

SimCLR (Simple Framework for Contrastive Learning) – метод самообучения, сближающий представления двух аугментированных видов одного изображения

softmax – нормализация вектора логитов в распределение вероятностей по классам

SSD (Single Shot Detector) – однопроходный детектор объектов, предсказывающий рамки и классы напрямую из карт признаков

Starlette – асинхронный ASGI-фреймворк для Python, лежащий в основе FastAPI

SVM (Support Vector Machine) – метод опорных векторов для классификации и регрессии

Swagger UI – веб-интерфейс интерактивной документации REST API

Swin Transformer – иерархический визуальный трансформер с локальным вниманием в окнах

TensorBoard – инструмент визуализации процесса обучения нейросетей (кривые потерь, метрики, гистограммы весов)

timm (PyTorch Image Models) – библиотека архитектур и предобученных весов для компьютерного зрения на PyTorch

U2-Net – CNN для плотной сегментации и выделения объектов

UNet – сверточная сеть в схеме кодер-декодер

Uvicorn – ASGI-сервер для запуска асинхронных Python-приложений

VGG – семейство глубоких CNN с однородными свертками, предложенное группой Visual Geometry Group

ViT (Vision Transformer) – модель, режущая изображение на части и использующая самовнимание

XGBoost – реализация градиентного бустинга над решающими деревьями

YOLO (You Only Look Once) – семейство однопроходных детекторов объектов в режиме реального времени

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

Ансамбль моделей – несколько моделей с объединением ответов

Атрибут предмета одежды – дискретная характеристика вещи из конечного перечня

Аугментация данных – расширение обучающей выборки преобразованиями без смены метки

Базовый цвет – метка доминирующего цвета из фиксированного списка

Валидационная выборка – данные для оценки качества и подбора гиперпараметров в ходе обучения

Веб-приложение – программа, с которой работают через браузер

Векторное представление (эмбединг) – числовой вектор фиксированной длины, в который кодируют изображение или другой объект нейросетью

Гиперпараметр – величина, задаваемая до обучения и не обновляемая шагом по градиенту

Глубокое обучение – машинное обучение на многослойных нейросетях, где признаки извлекаются из данных в ходе обучения, а не задаются вручную

Голова модели (head) – финальные слои, формирующие ответ задачи по внутреннему представлению

Голосующий классификатор (VotingClassifier) – ансамблевый классификатор, объединяющий предсказания нескольких базовых моделей голосованием по меткам или усреднением вероятностей

Градиентный бустинг – последовательное добавление слабых моделей под ошибку текущего ансамбля

Двунаправленная LSTM – рекуррентная нейросеть, обрабатывающая последовательность в прямом и обратном направлении

Доля верных ответов (accuracy) – доля правильно классифицированных объектов среди всех объектов

Дообучение (fine-tuning) – донастройка весов модели, уже обученной на другой задаче или корпусе, на целевых данных

Извлекатель признаков (backbone) – часть сети, строящая вектор признаков по изображению

Классификация – выбор одного класса из заданного набора по описанию объекта

Конечная точка (endpoint) – URL и HTTP-метод для одной операции API

Контейнер – изолированный пакет приложения с зависимостями для переносимого запуска

Косинусная близость – косинус угла между векторами как мера сходства на отрезке от -1 до 1

Кросс-энтропия – функция потерь многоклассовой классификации по вероятностям классов

Логистическая регрессия – линейный классификатор, превращающий через сигмоиду взвешенную комбинацию признаков в вероятность принадлежности к классу

Макроусреднение – среднее метрики по классам с равными весами

Мера Хэмминга (Hamming loss) – доля позиций вектора меток, предсказанных неверно; применяется в многозначной классификации

Метрика качества – числовая оценка близости предсказаний модели к истинным меткам

Микросервис – компонент с узкой ответственностью, развертываемый независимо от остальной системы

Модель машинного обучения – параметрическое отображение вход-выход с весами, найденными обучением

Мягкое голосование (soft voting) – усреднение вероятностей по классам от нескольких классификаторов

Набор данных – коллекция объектов с разметкой или без для обучения и проверки

Нормализация – приведение признаков к согласованному масштабу для устойчивого обучения

Обучающая выборка – данные, на которых считают градиенты и обновляют веса

Обучающая эпоха – один полный проход по обучающей выборке

Остаточные связи (skip connections) – соединения, передающие активации в обход одного или нескольких слоев и суммирующие их с выходом, позволяют строить сети в сотни слоев без затухания градиентов

Оценка совместимости – числовая мера сочетаемости выбранных вещей в комплекте

Пакетная нормализация – нормализация активаций по маленькому пакету внутри слоя

Переобучение (overfitting) – малая ошибка на обучающей выборке и заметно худший результат на новых данных из-за подгонки к шуму

Полнота (recall) – доля правильно предсказанных положительных примеров среди всех истинно положительных объектов

Предобработка данных – подготовка сырых данных к входу модели (формат, размер, фильтрация)

Предобученная модель – модель с весами с другой задачи как начальное приближение для целевой

Предсказание – выход модели (метка, число, вектор) на заданном входе

Признак (feature) – одна измеримая характеристика объекта на входе модели

Признаки Хаара – прямоугольные фильтры, вычисляющие разность яркостей соседних областей изображения, применялись в классических детекторах объектов

Пространство признаков – линейное пространство, где объект задан набором числовых признаков

Ранжирование – упорядочивание объектов по выбранному показателю

Ранняя остановка (early stopping) – остановка обучения при отсутствии улучшения на валидации за заданное число эпох

Сверточная нейронная сеть – нейросеть на основе сверток для изображений и сеточных данных

Сегментация изображения – разметка областей кадра классами объекта, части или фона

Сид (seed) – начальное значение генератора псевдослучайных чисел, фиксирующее воспроизводимость случайных событий в эксперименте

Случайный лес – ансамбль решающих деревьев, обученных на случайных подвыборках данных, итоговый ответ определяется большинством голосов или усреднением предсказаний

Составной HTTP-запрос (multipart/form-data) – формат HTTP-запроса, позволяющий передавать файлы и текстовые поля в одном сообщении

Тестовая выборка – данные только для итоговой оценки без участия в настройке модели

Тип-осведомленный подход (type-aware) – способ представления совместимости, при котором для каждой пары категорий предметов одежды используется свое подпространство признаков

Точность (precision) – доля правильно предсказанных положительных примеров среди всех предсказанных положительных

Трансформер – архитектура, основанная на механизме внимания к последовательности, адаптированная к изображениям

Уверенность модели – вероятность предсказанного класса или родственная величина вместе с меткой

Узкое место (bottleneck) – блок с промежуточным слоем уменьшенной размерности

ВВЕДЕНИЕ

В повседневной жизни учет одежды редко укладывается в пару полок: в шкафу часто десятки и сотни позиций. Сезонная перестановка и новые покупки меняют фактический набор, а вопрос «что надеть» возвращается каждый день. Более того, на фоне избытка предложения в рознице и укороченных циклов обновления коллекций заметнее становится задача рационального использования уже имеющегося: снижение повторных покупок, прозрачность запаса и ускорение повседневного выбора одежды без полного перебора шкафа.

Бытовой учет содержимого гардероба нередко остается разрозненным: память, бумажные списки и несогласованные файлы, которые расходятся с фактическим составом после покупок и сезонных перестановок, а подбор образа под настроение, погоду и повод снова упирается в ручной перебор шкафа. Рядом с этим в других делах уже привычны цифровые списки и напоминания, и возникает запрос на такое же решение для своих вещей: видеть состав гардероба на экране и получать подсказки по комплектам на каждый день.

Именно поэтому в рамках выпускной квалификационной работы в формате бизнес-тезиса создано веб-приложение «Plechiki». В нем пользователь переносит личный гардероб в цифровой учет и ведет карточки вещей, а подборки образов строятся только из того набора предметов, который он уже завел в систему.

Проект реализуется командой из 5 человек: архитектор, frontend-разработчик, backend-разработчик, аналитик и ML-инженер. Автор отвечает за ML-составляющую – от подготовки данных до развертывания моделей.

Цель работы – повышение релевантности рекомендаций по подбору одежды посредством разработки и интеграции модуля машинного обучения, решающего задачи классификации атрибутов и оценки визуальной совместимости элементов гардероба.

Задачи работы сводятся к следующему:

- 1) Провести исследование предметной области и анализ существующих решений в области цифрового управления гардеробом.
- 2) Выполнить сбор и подготовку наборов данных для задач классификации атрибутов одежды и оценки совместимости элементов гардероба.
- 3) Провести предобработку данных, включая очистку, преобразование признаков и формирование обучающей и тестовой выборок.
- 4) Провести экспериментальное исследование моделей машинного обучения для задачи многоклассовой классификации атрибутов одежды.
- 5) Провести экспериментальное исследование моделей для задачи оценки совместимости элементов гардероба.
- 6) Выполнить сравнительный анализ алгоритмов и выбрать оптимальные модели на основе показателей качества.
- 7) Разработать алгоритм формирования рекомендаций образов на основе результатов классификации и оценки совместимости элементов гардероба.
- 8) Интегрировать разработанный модуль машинного обучения в веб-приложение.

Практическая значимость проекта «Plechi» заключается в том, что рутинные шаги учета и перебор удачных сочетаний вещей уходят из чисто ручного режима и опираются на сценарии сервиса. Учет одежды ведется в одной структуре данных, без параллельных заметок и опоры на память, а комплекты собираются из того, что уже лежит в «цифровом гардеробе». Это укорачивает хаотичный перебор при выборе одежды и снижает риск повторной покупки вещи с одинаковой функцией.

1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ АНАЛИЗА ГАРДЕРОБА С ИСПОЛЬЗОВАНИЕМ МЕТОДОВ МАШИННОГО ОБУЧЕНИЯ

В главе выстроена теоретическая база для последующих экспериментов. Первые два раздела фиксируют проблему: чем управление личным гардеробом затруднено вручную и где существующие цифровые сервисы оставляют пробел. Следующие разделы разбирают задачи компьютерного зрения для одежды и эволюцию архитектур – от классических дескрипторов до трансформеров, – подходы к оценке совместимости комплектов, метрики и инструменты, выбранные для реализации. Эта картина определяет постановку задач и выбор архитектур в экспериментальных частях работы.

1.1 Анализ проблемы управления персональным гардеробом

Учет и использование личного набора одежды – обычная практика, но масштаб рынка и темп смены коллекций сделали ее заметно тяжелее. Дешевизна производства, онлайн-торговля и логика быстрой моды подняли объем выпуска: по оценке Ellen MacArthur Foundation, с 2000 по 2015 год мировое производство одежды более чем удвоилось, а средний срок носки единицы сократился примерно на 36 процентов [1]. В итоге у многих людей шкаф переполнен, а реально носится лишь часть вещей.

Опросы в исследованиях потребительского поведения дают порядок величины: в активной ротации обычно оказываются лишь 20-40 процентов имеющегося гардероба, остальное носят редко или не носят [2]. Это называют «эффектом переполненного шкафа». Причины смешанные: нет ясного списка того, что уже куплено, вещи «теряются» из виду, сложно быстро собрать комплект, отдельные покупки плохо стыкуются по стилю, цвету или сезону. Дальше часто идет покупка дубликата по функции, и круг замыкается.

Здесь есть и бытовая проблема времени и внимания. Ежедневный выбор одежды – серия мелких решений при большом числе вариантов. При ограниченном запасе внимания повтор таких решений дает известную в психологии усталость от выбора и ухудшает качество последующих решений

в течение дня [3]. Если гардероб не разложен по полкам хотя бы в голове или в записи, утренний подбор образа растягивается.

Экологический слой тоже нельзя снять со счетов: текстиль – ресурсоемкая отрасль с заметным вкладом в загрязнение. По данным Минприроды РФ и Росстата, в стране ежегодно накапливаются сотни тысяч тонн текстильных отходов, а доля переработки остается невысокой [4]. Покупки «с запасом» на фоне слабого использования того, что уже есть, увеличивают массу отходов и углеродный след личного потребления.

Сводка проблем выглядит следующим образом:

- 1) редко бывает целостная и обновляемая картина состава гардероба;
- 2) неудобно вести вещи по атрибутам – категория, цвет, материал, сезонность, стиль;
- 3) при большом числе комбинаций трудно вручную искать сочетаемые комплекты;
- 4) почти не фиксируется, что надевали и как часто, поэтому плохо видно, что простаивает и что дублирует другое.

Одними бытовыми привычками и личными записями такой объем плохо закрывается: данных много, а оценки частично субъективны. Отсюда интерес к цифровому учету и подсказкам по комплектам. Машинное обучение в таких продуктах снимает часть ручного ввода и дает возможность опереться на более богатый набор визуальных и смысловых признаков, чем позволяют только ручные записи.

1.2 Анализ цифровых сервисов управления гардеробом

За последнее десятилетие сложился отдельный пласт мобильных и облачных приложений под учет личного гардероба. Обычно там есть цифровой каталог, сборка образа на экране, иногда учет носки и редко рекомендации комплектов. Набор функций похож, а вот технологическое решение расходится: где-то почти все сводится к ручному вводу, где-то подключают компьютерное зрение и рекомендательные блоки. Ниже

разобраны несколько типичных линеек продуктов, чтобы зафиксировать, где у сегмента остаются ограничения.

Приложение Stylebook – пример с упором на ручную кураторскую работу: фото вещи, отделение фона, своя разметка, перетаскивание карточек в образ и календарь носки [5]. Точность цифрового двойника шкафа здесь упирается во время пользователя: атрибуты и комплекты в основном собираются без заметной автоматизации.

Другой уровень дают приложения Acloset, Whering и Indyx. Фон и черновая категоризация часто опираются на методы компьютерного зрения, поэтому добавление новой вещи обходится меньшим числом действий [6]. У Acloset встречается автоматическая генерация комплектов из содержимого гардероба с учетом погоды и типа события. У Whering есть рекомендации с учетом стилевых предпочтений и социальные функции обмена образами внутри платформы.

Отдельно стоят связки «гардероб и подсказка к покупке». Платформы Finery и Cladwell в прошлом активно связывали личный шкаф с каталогами ритейлеров и предлагали докупить то, чего не хватает, чтобы замкнуть набор комплектов [7]. Для магазинов это понятная модель, но с точки зрения сокращения излишнего потребления она слабая: сервис чаще подталкивает к новой покупке, а не помогает обойтись тем, что уже куплено.

По глубине машинного обучения ландшафт неоднороден. Удаление фона через сегментацию общего назначения – почти норма. Автоматические атрибуты – категория, цвет, фасон, материал – встречаются с разным качеством и нередко требуют правки вручную. Оценка совместимости и сбор стилистически цельного комплекта либо отсутствуют как задача, либо держатся на коротком наборе эвристик вроде типа вещи и базовой цветовой совместимости.

Ручной ввод все еще занимает заметную долю сценария. Классификация атрибутов часто не дотягивает до уровня, где рекомендации можно не проверять. Совместимость редко учится на данных и почти не выходит за пару

простых правил. Также много продуктов заточено под англоязычный контекст ассортимента и стиля, который для части пользователей в других регионах просто не совпадает с реальностью магазинов.

На этом фоне оправдана разработка собственного решения с более плотным контуром ML (Machine Learning): и на этапе распознавания атрибутов по фотографии, и на этапе подбора комплектов. Имеет смысл создавать модель совместимости, которая не будет сводиться к двум правилам из учебника колористики – тогда нагрузка на пользователя упадет, а практическая отдача от системы возрастет.

1.3 Методы машинного обучения для распознавания элементов одежды

Разбор фотографий одежды в компьютерном зрении давно выделился в отдельный пласт задач: из кадра нужно получить структурированное описание вещи или комплекта. Поток визуального контента из электронной торговли и социальных сетей снабдил исследования и продукты крупными корпусами снимков, что стимулировало прикладные сценарии – от визуального поиска и виртуальной примерки до рекомендаций и цифрового гардероба. Удобно разложить постановки по тому, что именно считается ответом: одна метка класса, рамка, маска, набор атрибутов или близость в пространстве признаков.

Самая прямая постановка – одноклассовая классификация кадра: модель выбирает одну категорию из фиксированного списка вроде «платье», «брюки», «футболка», «верхняя одежда». Для одежды внутриклассовая вариативность высокая: фасон, ткань, ракурс, складки и часто целый человек в кадре сдвигают распределение признаков. Поэтому обычно идут в сторону глубоких сверточных сетей и архитектур с механизмом внимания [8].

Если на снимке несколько вещей – типичный случай фото в полный рост или готового образа, – разумнее использовать детекцию: для каждого экземпляра ищут ограничивающую рамку и класс. Так можно вырезать элементы и обрабатывать их по отдельности. В линейке практичных семейств

долго держались R-CNN (Region-based Convolutional Neural Network), SSD (Single Shot Detector) и YOLO (You Only Look Once). Позже подключились трансформерные детекторы вроде DETR (DEtection TRansformer) [9].

Сегментация – семантическая или по экземплярам – дает маску на уровне пикселей и точнее фиксирует границу предмета. Это нужно и для виртуальной примерки с учетом силуэта, и для аккуратного отделения фона, когда вещь кладут в личный каталог. Под задачу существуют специализированные модели и датасеты с плотной разметкой, включая Mask R-CNN, универсальные сегментаторы нового поколения [10] и коллекции вроде ModaNet [11].

Отдельная линия – атрибуты: цвет, материал, длина рукава, вырез, узор, сезон и другие. Здесь ответ многометочный. Так как одной метки мало, чаще строят несколько голов или многозадачную схему. Крупные открытые наборы DeepFashion и DeepFashion2 как раз заточены под совместную разметку категорий и длинного списка атрибутов [12].

Поиск «похожих» вещей в коллекции опирается на эмбединги: изображение отображают в вектор, а дальше сравнивают расстоянием или косинусом. Качество такого пространства напрямую сказывается на рекомендациях и на численной оценке сочетаемости. Далее обучают представления метрическими методами, контрастивными потерями и парами «согласовано / не согласовано» [13].

Для персонального гардероба в центре оказываются связка «категория и атрибуты» и вектор, пригодный для последующей модели совместимости. Сегментация в таком контуре чаще вспомогательная: ее типично используют на предобработке, прежде всего для удаления фона.

Под задачи классификации категорий и атрибутивной разметки в компьютерном зрении сложился обширный набор архитектур – от ручных дескрипторов до трансформеров. Классификация изображений – базовая постановка компьютерного зрения: на входе картинка, на выходе одна метка из заранее зафиксированного списка. Классическую цепочку обработки долгое

время строили из HOG (Histogram of Oriented Gradients), SIFT (Scale-Invariant Feature Transform), признаков Хаара и дальше обучали SVM (Support Vector Machine), k-NN (k-Nearest Neighbors) или случайный лес [14]. Это работало, пока под задачу удавалось подобрать дескриптор. Перенос на новый домен часто означал новый цикл проектирования признаков.

Глубокие сверточные сети сменили описанный порядок тем, что признаки больше не проектируют отдельным конвейером: их иерархию выстраивают обучаемые фильтры прямо по данным. Масштабную проверку этому принципу дала AlexNet, победившая на конкурсе по компьютерному зрению на наборе ImageNet [15]. За ней пошли более глубокие и модульные VGG, GoogLeNet и Inception, но рост числа слоев вскрыл затухание градиентов и создал потолок на «простое» удлинение сети.

ResNet (Residual Network) обошел узкое место остаточными связями (skip connections), пропуская сигнал мимо части слоев и позволяя считать сети на сотни уровней [16]. Дальше инженерный фокус сместился в сторону удешевления счета при близком качестве: EfficientNet с согласованным масштабированием глубины, ширины и разрешения входа [17] и MobileNet [18], а в 2022 году ConvNeXt подтянул к сверткам приемы из трансформеров – крупные ядра, иной bottleneck, LayerNorm (Layer Normalization) – и показал сопоставимое с трансформерами качество при более скромной цене вычислений [19]. ConvNeXt V2 закрепил этот результат, сочетая свертки с маскированным предобучением [20].

Параллельно в компьютерное зрение пришел механизм внимания из NLP (Natural Language Processing). Vision Transformer (ViT) показал, что «чистое» внимание на патчах тянет классификацию на уровень сильных CNN (Convolutional Neural Network), если данных на обучение хватает [21]. DeiT [22] снял ограничение на объем обучающих данных за счет дистилляции знаний. Семейство Swin Transformer добавило иерархию и локальные окна, смешивая идеи сверток и глобального внимания [23, 24].

На практике редко учат модель «с нуля» на маленькой выборке. Чаще берут веса с ImageNet или расширенного корпуса ImageNet-21K [25] и дообучают под свою разметку – это сокращает время экспериментов [26]. Для одежды связка «предобучение и тонкая настройка» давно стала нормой.

Отдельно выросло обучение с самоконтролем: SimCLR (Simple Framework for Contrastive Learning), MoCo (Momentum Contrast) [27] и родственные схемы, MAE (Masked Autoencoder) с восстановлением скрытых патчей [28], BEiT (BERT Pre-Training of Image Transformers) с восстановлением дискретных визуальных токенов [29], DINO (Distillation with No Labels) и DINOv2 [30, 31], а также CLIP (Contrastive Language-Image Pre-Training), где совместно обучают визуальный и текстовый кодировщики на парах «изображение-текст» [32]. Такие модели дают универсальные эмбединги, которые потом настраиваются под классификацию, поиск или мультимодальные сценарии.

Для классификации снимков одежды выбор архитектуры все еще сводится к компромиссу между точностью, вычислительным бюджетом и объемом разметки. Сильные CNN вроде ResNet и EfficientNet и трансформерные семейства остаются рабочими кандидатами при аккуратной предобработке и переносе обучения. Кандидатные архитектуры и результаты их сравнения рассматриваются в третьей главе.

1.4 Подходы к оценке совместимости элементов гардероба

Совместимость – это не «похожесть картинок». Вещи могут отличаться по фасону и категории, но вместе выглядеть собранно. В расчет обычно входят тип предмета, цветовые связки, стиль, материал, сезон и даже контекст события. Пространство признаков растет, а вкус частично субъективен, поэтому чистая ручная формула редко закрывает задачу – чаще подключают обучение на данных.

Самый старый метод – эвристики из стилистики и колористики: круг Иттена, комплементарные и родственные схемы, простые правила про

формальность и силуэт [33]. Реализовать легко, но охват узкий: фактура, принт и актуальные тренды в такую схему почти не попадают.

Когда появились большие коллекции пользовательских комплектов, стало проще учить модель на парах и наборах. Часто используемый корпус – Polyvore: десятки тысяч образов, у каждого набор карточек вещей [34]. Это дало обычное обучение с учителем, где совместимость трактуют как отношение между предметами или целым набором.

Сиамские схемы учат эмбединг так, чтобы близкие векторы отвечали сочетаемым вещам, а далекие – несочетаемым. Тренируют контрастом или триплетами «якорь-позитив-негатив». Есть варианты type-aware: отдельные подпространства под пары категорий, потому что закономерность «верх-низ» и «обувь-брюки» разная [35].

Образ можно трактовать как последовательность и прогнать через двунаправленные LSTM (Long Short-Term Memory): модель учится закрывать пропуск в комплекте, и совместимость выучивается косвенно [36]. Плюс – видно сразу несколько предметов. Минус – нужен выбранный порядок вещей, а он не всегда естественен.

Графовые сети кладут вещи в вершины, а связи совместимости – в ребра. Семейство NGNN (Node-wise Graph Neural Network) хорошо показало себя на Polyvore и оказалось удобным для комплектов разной длины [37]. Рядом стоят внимание и трансформеры над множеством предметов без жесткого порядка.

Отдельный заход – старт с мультимодального предобучения вроде CLIP: взять уже богатые визуально-текстовые эмбединги и доучить под комплекты или встроить в граф [38]. Разметки на целевой задаче тогда нужно меньше.

В терминах архитектуры получают два полюса. В одном каждая вещь получает вектор, а совместимость считают из расстояний или скалярных продуктов. В другом модель сразу принимает целый набор – рекуррентно, через граф или через внимание.

1.5 Метрики оценки качества моделей

Сравнивать модели без чисел бессмысленно: нужны показатели, которые совпадают с постановкой задачи и с тем, как устроены классы в данных. В этой работе три разных типа задач – одноклассовая или многоклассовая классификация по категории, многозначная разметка атрибутов и оценка совместимости вещей в комплекте. Под каждый тип метрики подбирают отдельно. Сильный перекося классов сразу сдвигает акцент с «просто ассурасы» в сторону устойчивых к дисбалансу величин.

Доля верных ответов (ассурасы) – самая читаемая метрика, но на несбалансированных выборках ее легко увеличить доминирующим классом. Тогда смотрят точность, полноту, F1 по каждому классу и усредняют: макро-F1 дает равный вес редким меткам, микро-F1 ближе к доле правильно угаданных примеров с учетом их частоты [39].

Если порог решения не зафиксирован заранее, строят ROC (Receiver Operating Characteristic) и кривую precision-recall, а сводят их в ROC-AUC (Receiver Operating Characteristic Area Under Curve) и PR-AUC (Precision-Recall Area Under Curve). На сильном дисбалансе PR-AUC обычно говорит больше, ROC-AUC иногда выглядит слишком оптимистично. В многоклассовых схемах с длинным списком категорий дополнительно смотрят топ-k ассурасы: попала ли истинная метка в k самых вероятных ответов.

Атрибуты у вещи идут пачкой, это многозначная классификация. Типичный интегральный показатель – mean Average Precision (mAP), усредненный по атрибутам. Рядом держат Hamming loss (доля ошибочных битов меток), а также precision, recall и F1 в макро- и микроварианте по всему набору меток [40].

Эмбединги для поиска и совместимости оценивают как ранжирование. Recall@k считает, как часто правильный объект попадает в k ближайших соседей в пространстве признаков. Если важен порядок в списке, подключают NDCG (Normalized Discounted Cumulative Gain), штрафую низкие позиции релевантных элементов [41].

Для комплектов одежды в литературе на Polyvore закрепились свои цифры. FITB (fill-in-the-blank) accuracy – доля задач, где модель угадала пропуск в образе, выбрав один согласованный предмет среди отвлекающих вариантов [42]. Также есть и попарные метрики, где проверяют, правильно ли упорядочены пары по степени сочетаемости.

Однако качество на валидации – еще не весь ответ. Считают задержку на один запрос, пропускную способность при пакетной подаче, занятую оперативную память под веса и размер файла модели на диске. Эти величины показывают, укладывается ли архитектура в ресурсный контур сервера без постоянного ручного масштабирования.

1.6 Используемые инструменты и библиотеки

ML-модуль проходит цепочку от сырых файлов до HTTP-ответа в рабочей среде, и под каждый шаг нужен свой слой программного обеспечения: чтение и правка изображений, обучение, логирование, упаковка сервиса.

Язык реализации – Python 3.11. На нем держится большая часть экосистемы компьютерного зрения и табличных конвейеров обработки данных, удобно стыковать C/C++-ядра тензорных библиотек, а серверную часть ML-сервиса пишут в том же языковом окружении, без лишних адаптеров между языками [43].

Глубокое обучение ведется в PyTorch: динамический граф облегчает отладку и перебор идей, есть распределенное обучение и путь к ускоренному применению модели на выходе. Рядом стоит torchvision – базовые датасеты, типовые CNN и преобразования изображений [44]. Современные архитектуры (EfficientNet, ConvNeXt, Swin, ViT и другие) подтягиваются через timm (PyTorch Image Models) – один интерфейс и каталог предобученных весов [45].

OpenCV и Pillow закрывают низкоуровневую работу с пикселями. Отдельный контур удаления фона опирается на готовые предобученные модели. Albumentations дает быстрые аугментации на C-ядре – геометрия, цвет, композиции – и помогает удержать модель от переобучения на малой

выборке [46]. Автоматизированный подбор политики аугментаций закрывается адаптивными подходами вроде TrivialAugment [47].

Таблицы метаданных и разметки обрабатываются в pandas, массивы признаков – в NumPy. Классические классификаторы, конвейеры предобработки и стандартные метрики берутся из scikit-learn [48].

Эксперименты логируются в TensorBoard и Weights & Biases: кривые функций потерь, сравнение прогонов, привязка качества к гиперпараметрам, артефакты весов. Это облегчает повторяемость прогонов и отбор конфигурации [49].

HTTP-слой ML-модуля реализован на FastAPI: асинхронная связка Starlette и Uvicorn, автодокументация OpenAPI и проверка схем запросов и ответов на Pydantic [50]. Такой набор хорошо ложится на микросервис, который вызывается из клиентской части приложения.

Окружение фиксируется Docker-образом, чтобы зависимости не отличались между машинами. Для ускоренного применения модели в эксплуатации возможен экспорт графа из PyTorch в ONNX (Open Neural Network Exchange) и запуск через ONNX Runtime [51]. Версионирование проекта ведется через Git, проверки прогоняются через конвейер непрерывной интеграции. Описанный стек соответствует практикам, выявленным в анализе реальных сценариев развертывания ML-систем [52] и обзорах по MLOps [53].

1.7 Выводы по первой главе

Первая глава собрала в одну линию мотивацию, обзор рынка сервисов, теорию компьютерного зрения для одежды, совместимость комплектов, метрики и набор инструментов разработки. Ниже выводы, на которые опирается остальная работа.

Проблема личного гардероба не сводится к «неудобному приложению»: с ней связаны перерасход покупок, усталость от ежедневных мелких решений и внешние экологические издержки текстиля. Отсюда запрос на цифровой

учет, автоматизацию и помощь в сборке образов, а не только на очередной каталог вещей.

Более того, обзор цифровых сервисов показал типичный разрыв: много ручного ввода, ML чаще задействован точно (например, только фон), а совместимость вещей либо не моделируется всерьез, либо держится на коротких правилах. Для собственной системы это читается как требование – автоматизировать атрибуты по изображению и добавить обучаемую оценку сочетаемости комплектов.

По анализу одежды по фото в работе остаются три опорных блока: тип/категория, многозначные атрибуты и компактное векторное описание вещи для последующей совместимости. По методам классификации рационально опираться на глубокие CNN и трансформеры с переносом обучения, чтобы не требовать гигантской разметки под каждый новый сценарий. Метрики для последующих экспериментов согласованы с постановками: для атрибутов – precision, recall, macro-F1, mAP.

Для совместимости в литературе сложились два направления: в одном каждая вещь получает эмбединг, а близость в этом пространстве интерпретируется как сочетаемость; в другом модель принимает весь набор сразу – последовательно, графом или вниманием. Контрольные метрики для таких задач – FITB accuracy и ROC-AUC.

Инструментальная база: PyTorch и timm для глубоких экспериментов, scikit-learn для классических алгоритмов и метрик, FastAPI для HTTP-слоя ML-сервиса, Docker и при необходимости ONNX для воспроизводимости и ускорения вывода модели.

Первая глава сформировала теоретическую базу: разрыв в существующих сервисах обосновывает выбор задач, анализ архитектур определяет кандидатов для сравнения, а выбранный стек инструментов фиксирует среду для экспериментов.

Логичный следующий шаг – данные: откуда берутся снимки и комплекты, а также как их чистить, чтобы результаты обучения были как можно лучше. Этому посвящена вторая глава.

2 АНАЛИЗ И ПОДГОТОВКА ДАННЫХ

Глава охватывает все этапы работы с данными – от выбора источников до финального разбиения на выборки. Первые разделы описывают два набора данных, которые были выбраны для обучения – Kaggle Fashion Product Images для классификаторов атрибутов и Polyvore Outfits для модели совместимости. Следующие разбирают конвейер подготовки: фильтрацию и канонизацию меток под таксономию системы, сегментацию фона, аугментации при обучении. Завершают главу анализ итогового состава выборки и стратегия разбиения с учетом перекаса классов.

2.1 Источники и характеристика используемых данных

Классификация атрибутов опирается на снимки отдельных предметов с метками по типу, цвету, сезону и назначению. Оценка совместимости опирается на готовые комплекты: наборы вещей, которые вместе составляют один образ. Ниже описаны источники, подобранные для каждой группы.

Основной набор для атрибутов – Kaggle Fashion Product Images Dataset: каталог со структурированной категориальной разметкой. В нем свыше 44 тысяч каталожных снимков вещей сверху на ровном фоне, что ослабляет влияние лишнего контекста сцены на обучение [54]. У каждого изображения заданы тип, цвет, сезон, назначение. Для типа в работе используется 49 классов – от рубашек и брюк до платьев, пиджаков, верхней одежды и обуви. Набор выбран из-за размера, единообразия условий съемки и полноты меток. Примеры изображений показаны на рисунке 1.



Рисунок 1 – Примеры изображений из датасета для классификации атрибутов одежды. Источник: Составлено автором на основе источников [54]

Альтернатива – DeepFashion Category and Attribute Prediction: более 280 тысяч изображений, 50 категорий и 1000 атрибутов [55]. Лицензия ограничивает использование некоммерческими исследованиями, что расходится с целью промышленного внедрения системы. К тому же заметная часть кадров – одежда на человеке или вне студийных условий. Для сценария, где пользователь чаще фотографирует вещь отдельно, это добавляет сдвиг распределения. Поэтому DeepFashion в основную обучающую выборку не входил.

Для модели совместимости основной источник – Polyvore Outfits: десятки тысяч пользовательских образов с наборами согласованных вещей [56]. Более того, он часто встречается в публикациях на тему совместимости вещей, так что результаты проще сопоставить с литературой. Набор построен так, что у каждого образа есть идентификатор и у каждой вещи – изображение, текст и категория, как показано на рисунке 2, то есть можно опираться и на визуальные, и на текстовые признаки.



Рисунок 2 – Примеры изображений из датасета для обучения модели совместимости. Источник: Составлено автором на основе источников [56]

2.2 Формирование набора данных изображений одежды

Исходный Kaggle Fashion Product Images в сыром виде ограниченно пригоден для обучения: в метках встречаются разные строки для одной категории, часть кадров выходит за пределы нужной предметной области, классы распределены неравномерно. Чтобы получить рабочую выборку, собран конвейер из фильтрации, канонизации меток и отбора записей. Пороги и правила зафиксированы в JSON-конфиге политики набора данных, так что повторный прогон дает тот же результат.

На вход конвейера подаются таблица метаданных с категориальными атрибутами по каждому предмету и каталог файлов изображений. Для каждой строки таблицы проверяется наличие файла. Совпадения превращаются в промежуточные записи JSON (JavaScript Object Notation) строк, содержащие путь к снимку, сырые атрибуты, идентификатор источника. Такой слой удобен, если позже добавятся снимки из другого источника.

Дальше значения атрибутов приводятся к таксономии работы: допустимые множества для типа предмета, базового цвета, сезона и назначения. Варианты написания из исходной разметки приводятся к одному каноническому значению. Это убирает дубли единственного и множественного числа, расхождения регистра и расхождения в составных названиях. Строки, у которых после канонизации ключевые поля не попадают в допустимые множества, отбрасываются.

Одновременно с канонизацией действует доменный отбор под систему гардероба: остаются только верх, низ и обувь. Аксессуары, сумки, украшения, предметы личной гигиены и прочие категории отсекаются. Это сделано, чтобы обучение не тратилось на классы, которые приложение не использует.

Затем включаются фильтры качества изображения: не меньше 200 пикселей по ширине и высоте, оценка размытости через дисперсию лапласиана и порог по информационной энтропии. Также отсеиваются битые файлы, слишком однотонные кадры и снимки с низкой детализацией – то есть все, что ухудшит обучение при минимальной пользе.

Последний шаг обработки базового набора – отсечение редких классов по минимальному числу примеров.

Каждому прошедшему все этапы предмету присваивается `group_id` для последующего разбиения выборки без утечки связанных записей из обучающей части в тестовую. `group_id` строятся из идентификатора источника и исходного `id` предмета, поэтому при обновлении сырья границы групп сохраняются предсказуемо.

Выборка с выровненными метками, суженной предметной областью и отсеченными редкими классами поступает в предобработку изображений.

2.3 Предобработка изображений и подготовка обучающих выборок

Сырые снимки с разным фоном, лишними объектами в кадре, разными размерами и освещением ухудшают обучение классификатора. Поэтому визуальный вход приводится к единому виду: сначала автоматически убирается фон и записывается результат на диск, затем при сборке пакетов выполняется масштабирование и нормализация.

Вещи обрабатываются в два шага: фон удаляется моделью `rembg` – предобученным сегментатором на базе U2-Net [57], затем происходит обработка моделью `cloth UNet`, обученной на одежде и дающей более четкие границы вещи. Итоговая маска – пересечение двух: общая модель убирает фон, специализированная подрезает фрагменты тела и побочные объекты, с которыми `rembg` справляется хуже. Файл сохраняется как JPEG в каталог обработанных снимков, а в манифесте обновляется путь. Так обрабатывается весь набор. Результаты обработки представлены на рисунке 3.



Рисунок 3 – Примеры результатов предобработки изображений одежды с использованием rembg и cloth UNet. Источник: Составлено автором на основе источников [54, 57]

Вход приводится к квадрату 224×224 и по каждому каналу RGB вычитается среднее и делится на стандартное отклонение по правилам того же корпуса: у красного 0,485 и 0,229, у зеленого 0,456 и 0,224, у синего 0,406 и 0,225 (в каждой паре первое число – среднее, второе – стандартное отклонение). Константы совпадают с теми, что применяли при предобучении backbone, чтобы при дообучении не уводить распределение входов от привычного для весов масштаба.

Аугментация включается только на этапе обучения: увеличение с последующей обрезкой под 224×224 , горизонтальное отражение, сдвиг яркости, контраста и насыщенности до 20%, сдвиг оттенка до 3%. Вертикальное отражение, крупные повороты и искажения пропорций не используются – они размывают связь между видом предмета на снимке и категориальной меткой. На валидации и тесте остаются масштабирование до 224×224 и та же нормализация.

Для модели совместимости предобработка та же, но вход – не один предмет, а набор вещей в образе: каждая позиция обрабатывается тем же конвейером отдельно, затем собирается структура для сети. Обучающие пары строят из целого образа из исходного набора и варианта, где одну или несколько вещей подменяют случайными предметами той же категории из других комплектов. Так получаются примеры обоих классов бинарной задачи «согласован / не согласован» без сильного перекоса.

Сводка шагов представлена на рисунке 4.

Автообрезка полей прозрачности после сегментации показана в листинге 1. По альфа-каналу строят бинарную маску переднего плана с порогом, находят ограничивающий прямоугольник по ненулевым пикселям и обрезают кадр с запасом по краям.

Листинг 1 – Процедура автоматической обрезки прозрачных границ изображения. Источник: Составлено автором

```
def _autocrop_transparent_borders(
    img: Image.Image,
    alpha_threshold: int = ALPHA_THRESHOLD,
    padding_px: int = AUTO_CROP_PADDING_PX,
) -> Image.Image:
    rgba = img.convert("RGBA")
    alpha = rgba.getchannel("A")
    mask = alpha.point(lambda a: 255 if a > alpha_threshold else 0,
mode="1")
    bbox = mask.getbbox()
    if bbox is None:
        return rgba
    left, top, right, bottom = bbox
    left = max(0, left - padding_px)
    top = max(0, top - padding_px)
    right = min(rgba.width, right + padding_px)
    bottom = min(rgba.height, bottom + padding_px)
    return rgba.crop((left, top, right, bottom))
```

Отдельное сохранение сегментации и динамическое масштабирование, нормализация и аугментации в пакетах разделяют тяжелые шаги и вариативность обучения. В результате предобработки остаются каталог обработанных изображений и манифест меток. Дальше по ним строится разбор распределений классов.

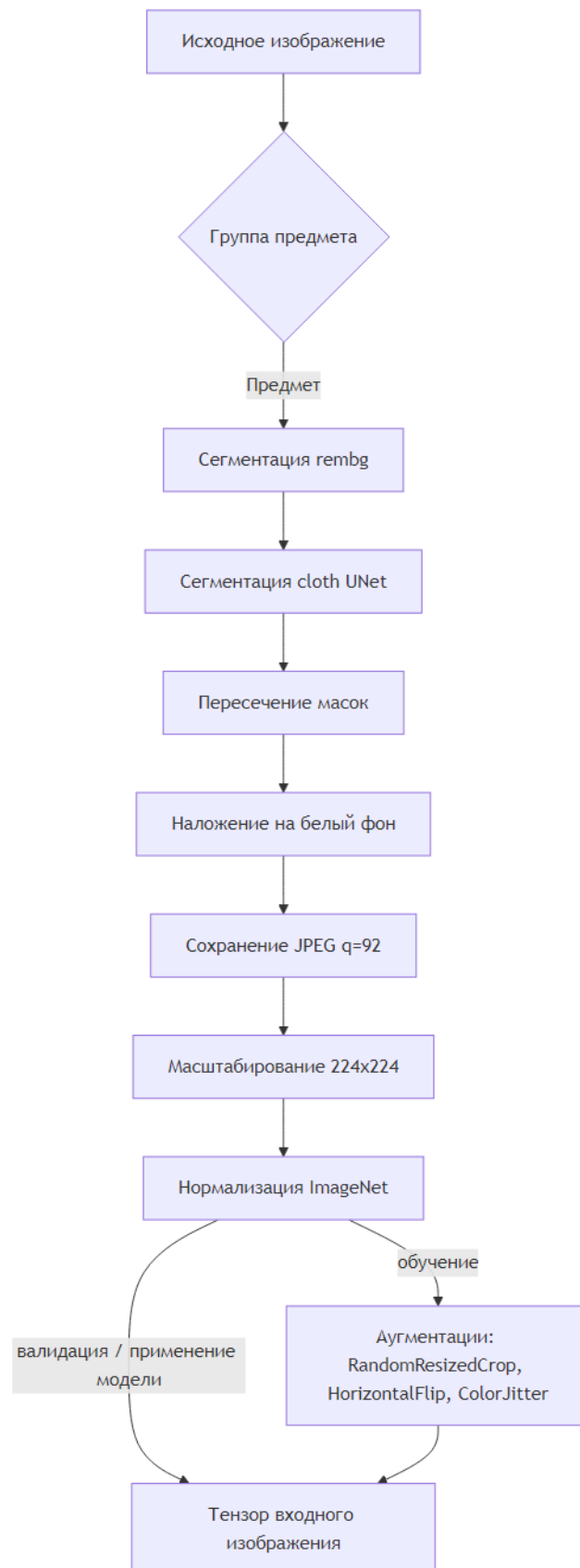


Рисунок 4 – Схема конвейера предобработки изображений одежды.
Источник: Составлено автором

2.4 Анализ распределения классов и характеристик данных

Перед обучением полезно зафиксировать, как классы и признаки картинок распределены в итоговой выборке: перекося по типам и цветам, связи между атрибутами, доли редких меток. Это подсказывает, нужны ли веса в функции потерь и иные приемы компенсации перекося классов. Ниже – срезы по основным категориальным полям и по геометрии исходных файлов.

Распределение по типу предмета показано на рисунке 5 и в таблице 1.

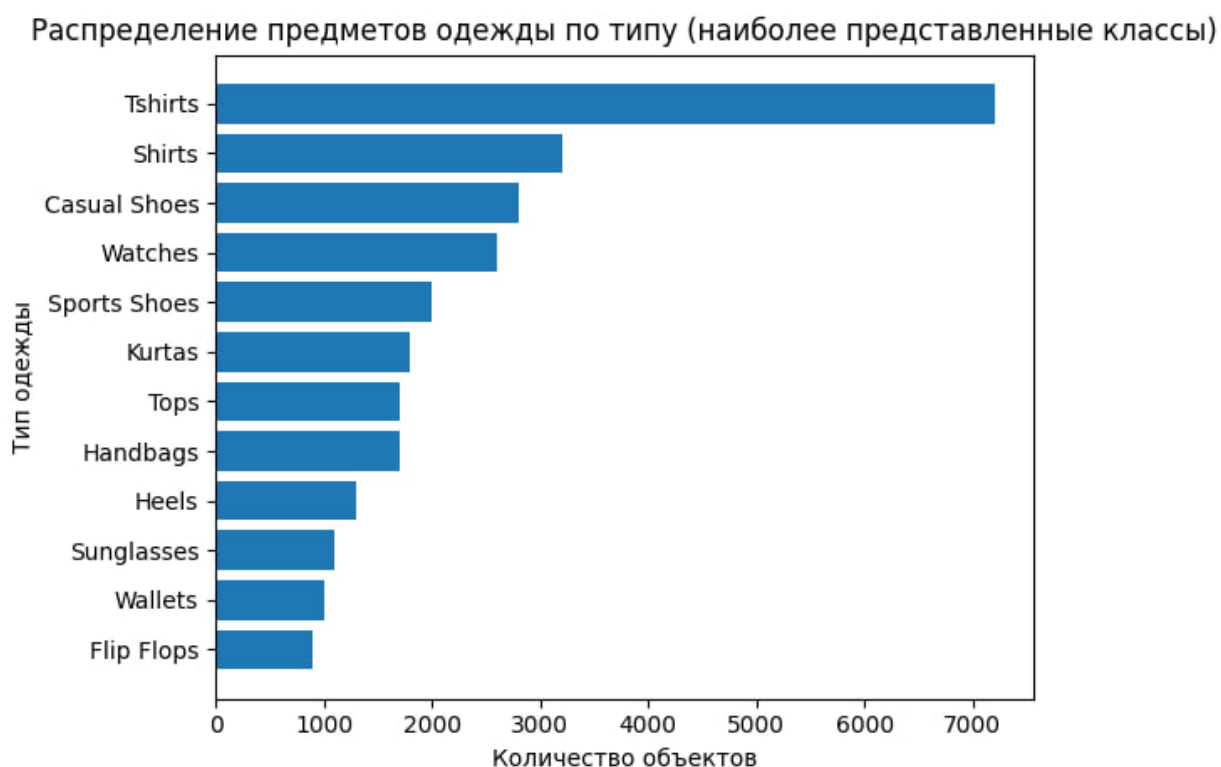


Рисунок 5 – Гистограмма распределения предметов одежды по типу (наиболее представленные классы). Источник: Составлено автором на основе источников [54]

Таблица 1 – Распределение предметов по типам (наиболее представленные классы). Источник: Составлено автором на основе источников [54]

Тип предмета	Группа	Число предметов	Доля в выборке, %
1	2	3	4
Tshirts	top	7 070	15,9
Shirts	top	3 218	7,2
Casual Shoes	footwear	2 846	6,4
Tops	top	1 762	4,0
Kurtas	top	1 844	4,1
Jeans	bottom	1 100	2,5
Sports Shoes	footwear	2 036	4,6
Heels	footwear	1 323	3,0
Formal Shoes	footwear	637	1,4
Dresses	top	464	1,0
Sweaters	top	274	0,6
Trousers	bottom	530	1,2
Shorts	bottom	547	1,2
Jackets	top	258	0,6
Track Pants	bottom	304	0,7
Sandals	footwear	507	1,1

1	2	3	4
Sports Sandals	footwear	131	0,3
Flip Flops	footwear	916	2,1
Прочие классы (31 тип)	—	около 3 000	6,7
Итого	—	около 28 800	100,0

Картина типична для товарного каталога: много базовых вещей повседневного спроса (футболки, рубашки, джинсы, повседневная обувь), мало узких категорий. Перекос усиливается самой структурой исходного Kaggle-набора.

Сводка по трем группам после доменной фильтрации (верх, низ, обувь) дана на рисунке 6. Три группы занимают сравнимые доли, с небольшим перевесом верха из-за большего числа различных типов внутри категории. Такое распределение стыкуется с разными ветками сегментации по группам предмета и с логикой сборки образа, где нужны вещи из каждой группы.

Распределение по цвету представлено на рисунке 7. Черный, белый, синий и серый заметно доминируют. Насыщенные оттенки (оранжевый, фиолетовый, бирюзовый и другие) встречаются реже. Для классификатора цвета в обучении используется взвешенная функция потерь, чтобы редкие оттенки сохраняли достаточный вклад в градиент.



Рисунок 6 – Гистограмма распределения предметов одежды по группам (верхняя одежда, нижняя одежда, обувь). Источник: Составлено автором на основе источников [54]

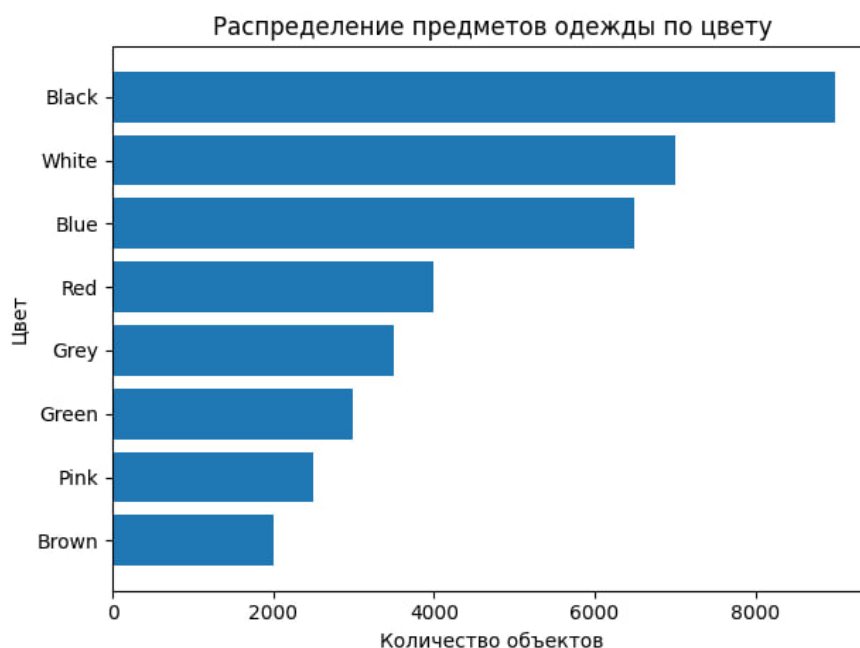


Рисунок 7 – Гистограмма распределения предметов одежды по значениям атрибута цвета в составе сформированного набора данных. Источник: Составлено автором на основе источников [54]

Сезон разбит на четыре класса, распределение показано на рисунке 8. Перекос слабее, чем у типа и цвета, но летних позиций чуть больше из-за

состава каталога. Назначение включает повседневное, спортивное, формальное и праздничное. Большая доля в распределении, изображенном на рисунке 9 – повседневное, что совпадает с реальной структурой ассортимента.

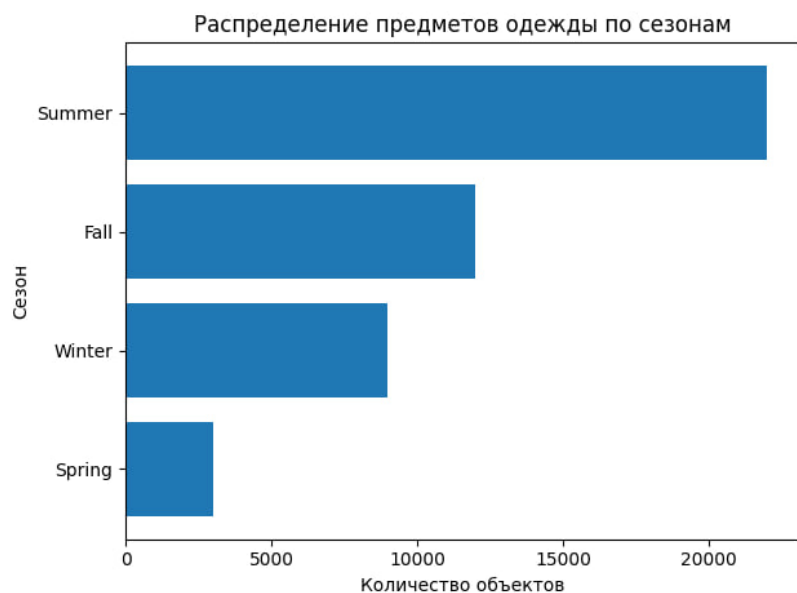


Рисунок 8 – Гистограмма распределения предметов одежды по значениям атрибута сезона в составе сформированного набора данных. Источник: Составлено автором на основе источников [54]

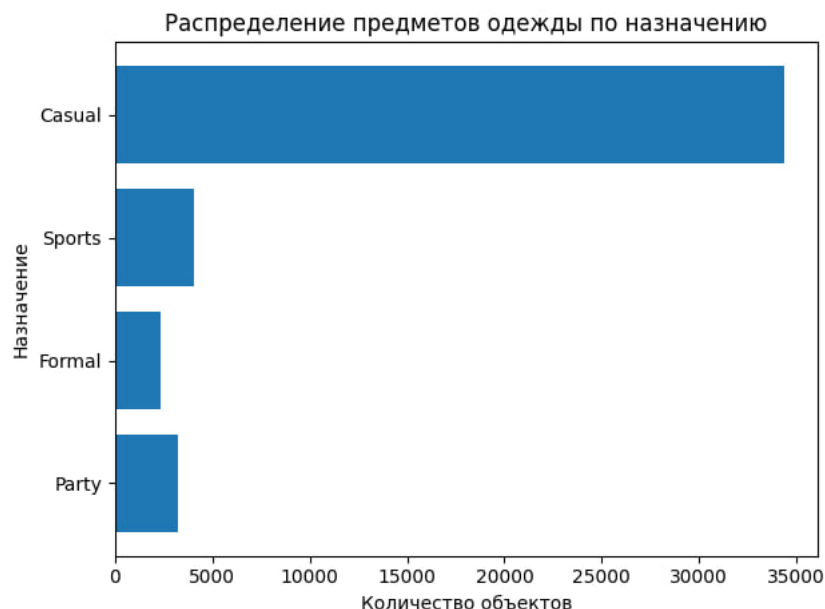


Рисунок 9 – Гистограмма распределения предметов одежды по значениям атрибута назначения в составе сформированного набора данных. Источник: Составлено автором на основе источников [54]

Совместные распределения показывают ожидаемые связи, например, как показано на рисунке 10. Сезон зависит от типа: у пальто, курток, свитеров чаще зима и осень, у открытой обуви (сандалии, шлепанцы) – лето. Тип связан и с назначением: пиджаки и формальная обувь тяготеют к формальному назначению, кроссовки и тренировочные комплекты – к спортивному.

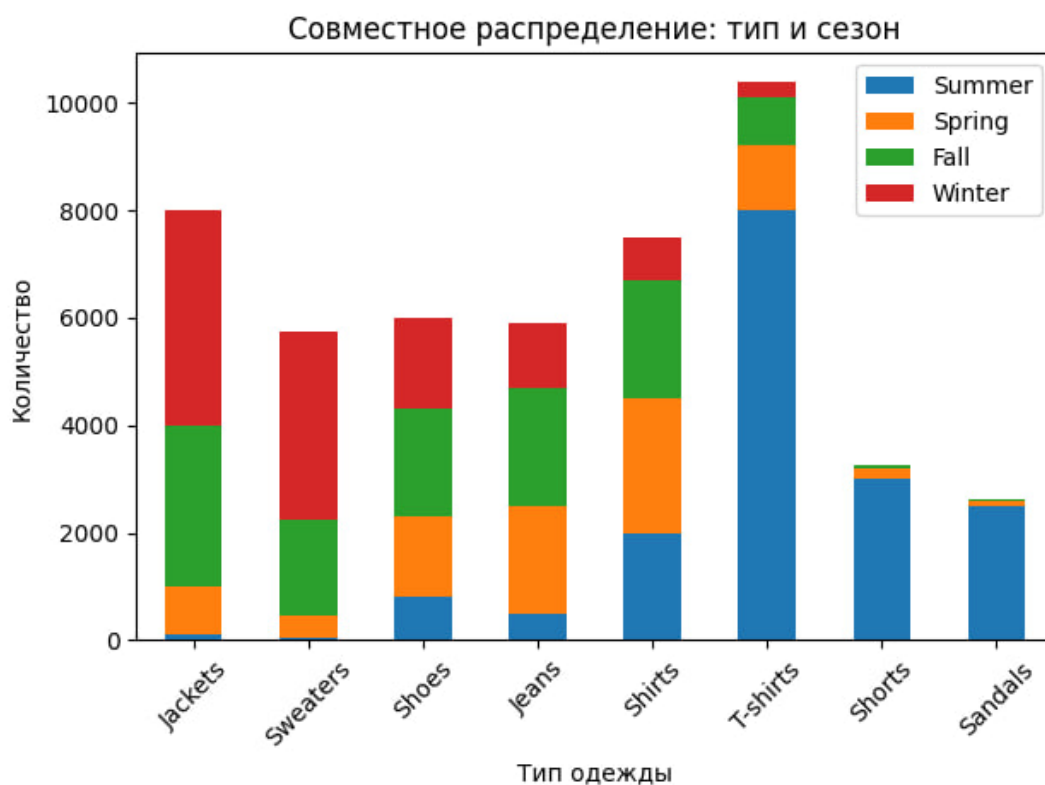


Рисунок 10 – Распределение типов одежды по сезонам в датасете. Источник: Составлено автором на основе источников [54]

Таблица 2 собирает число записей после каждого шага выравнивания таксономии и фильтрации и после предобработки изображений.

По строкам таблицы видно, какой этап отрезает сколько записей: основной спад дает доменный отбор и пороги по классам.

Таблица 2 – Объем выборки на этапах формирования набора данных.
Источник: Составлено автором на основе источников [54]

Этап обработки	Объем выборки предметов	Доля от исходного, %
Исходный набор Kaggle Fashion Product Images	44 441	100,0
После доменной фильтрации по группам верх/низ/обувь	32 187	72,4
После фильтрации по минимальному числу примеров в классах	29 124	65,5
После удаления записей с некорректными изображениями	28 815	64,8
После применения процедуры удаления фона	28 815	64,8
Итоговый объем сформированного набора данных	28 815	64,8

2.5 Разделение данных на обучающую и тестовую выборки

Честная оценка моделей возможна только при условии, что доли классов в обучающей, валидационной и тестовой частях близки к полной выборке.

Доли объема выборок выбраны следующие: 70% обучение и по 15% – валидация и тест. Такое разбиение часто берут для классификации изображений: остается достаточно примеров для обучения и при этом две

независимые контрольные части. Валидация идет на подбор гиперпараметров, раннюю остановку и промежуточные метрики. Тест используется один раз для итоговых чисел и не используется при настройке.

Стратификация идет по типу. При сильном перекосе типов, видимом на гистограммах выше, случайное деление без стратификации может выдать редкий класс из валидации или теста и сделать метрики шумными. Здесь доли каждого типа в трех частях совпадают с полной выборкой с точностью до одного примера на класс.

Параллельно учитывается `group_id`: все строки одного товара оказываются в одной части, иначе разные снимки или варианты разметки одной вещи окажутся и в обучении, и в тесте – это завышает качество. В реализации сначала применяется групповое разбиение со случайным перемешиванием, отделяя обучающую долю от временного хвоста, затем временный хвост стратифицированно делится на валидационную и тестовую подвыборки.

Начальное состояние генератора псевдослучайных чисел зафиксировано значением 42: при неизменном входе разбиение повторяется одинаково, что нужно для сопоставления экспериментов. Если меняются фильтры или состав источников, сплит пересчитывается и фиксируется в метаданных подвыборок.

В задаче совместимости единица разбиения – образ, а не один предмет. Идентификатор образа не должен пересекаться между обучающей и тестовой частями: иначе модель видела бы почти тот же комплект при обучении. Разбиение предметов согласуют с принадлежностью к образам, чтобы на тесте оставались и новые сочетания уже виденных вещей, и комплекты с ранее не встречавшимися предметами.

Итоговые объемы и число классов по подвыборкам – в таблице 3.

Таблица 3 – Распределение предметов одежды по подвыборкам. Источник: Составлено автором

Подвыборка	Обучающая	Валидационная	Тестовая	Итого
Доля, %	70	15	15	100
Число предметов	20 170	4 322	4 323	28 815
Число классов типа	49	49	49	49
Число классов цвета	18	18	18	18
Число классов сезона	4	4	4	4
Число классов назначения	4	4	4	4

Сверка долей классов по атрибутам в каждой части с полной выборкой подтверждает, что стратификация сработала. После отсеечения редких классов по порогу числа примеров у каждого оставшегося класса есть представители во всех трех частях, так что метрики на тестовой подвыборке считаются по каждому классу без пропусков.

2.6 Выводы по второй главе

Во второй главе собрана цепочка от сырых источников до разбитой выборки: откуда взяты данные для атрибутов и для совместимости, как производилась чистка и нормализация снимков, какие перекосы классов получились и как удалось разделить записи для честной проверки моделей.

Для классификации атрибутов опираются на Kaggle Fashion Product Images. Для совместимости комплектов – на Polyvore Outfits. Конвейер данных выравнивает метки по таксономии, оставляет только верх, низ и обувь, отбрасывает низкокачественные кадры и редкие классы по порогам, затем сегментирует фон и при обучении доводит изображения до 224×224 , нормализации ImageNet и набора аугментаций.

По итоговой выборке заметен перекося по типу и цвету. Поэтому разбиение на выборки строится со стратификацией по типу предмета и с учетом group_id, чтобы не смешивать связанные записи между обучением и контролем. Для образов совместимости граница проходит по идентификатору комплекта.

Подготовленные выборки честно отражают условия применения системы: доменный отбор, стратифицированное разбиение и задокументированные пороги фильтрации позволяют воспроизвести эксперименты. На этой базе строятся и сравниваются модели третьей главы: классификаторы атрибутов, модель совместимости и алгоритм формирования образов.

3 РАЗРАБОТКА И СРАВНИТЕЛЬНАЯ ОЦЕНКА МОДЕЛЕЙ

Данная глава посвящена разработке моделей. Первые разделы описывают постановку задач и архитектурные решения: четыре классификатора атрибутов – тип, цвет, сезон, назначение – и бинарный классификатор совместимости комплектов по триплету верх-низ-обувь. Совместимость комплекта сводится к бинарной задаче, а атрибутные метки участвуют в подборе комплектов и в карточке вещи при добавлении фото. Далее описаны процедура обучения и подбора гиперпараметров, сравнение одиннадцати архитектур-кандидатов по F1 и алгоритм подбора комплектов из пользовательского гардероба.

3.1 Разработка модели классификации атрибутов одежды

Существует несколько путей решения задачи классификации. Один путь – несколько классификационных голов на одном общем извлекателе признаков: общая сверточная или трансформерная часть и отдельная голова на каждый атрибут. Меньше суммарный объем весов, проще переиспользовать представление между похожими задачами. Другой путь – отдельная полная модель под каждый атрибут. Тогда независимо подбирается сеть и гиперпараметры и проще выжать качество на сложном атрибуте.

В работе выбран вариант с отдельной моделью на каждый атрибут. Атрибуты сильно различаются: от 4 до 49 классов, разные распределения в подготовленных выборках, разный визуальный сигнал – геометрия и силуэт для типа, цветовые каналы, более контекстные признаки для сезона и назначения. Поэтому четыре отдельные сети остаются практичным вариантом. Отдельные файлы весов еще и упрощают сопровождение: обновляем, скажем, только цвет, не трогая остальные.

Базовый блок – сверточная или трансформерная извлекающая часть с полносвязной классификационной головой на softmax по классам данного атрибута; альтернативой в многозначных задачах служат масштабируемые

декодерные головы [58]. Извлекатель стартует с весов ImageNet. Голова инициализируется случайно и доучивается вместе с извлекателем на выборке одежды. Так уменьшается нужный объем разметки и время сходимости по сравнению с обучением с нуля.

Для отбора извлекателя признаков в сравнение включены одиннадцать архитектур: ResNet-18, ResNet-34, ResNet-50, EfficientNet-B0, EfficientNet-B1, MobileNetV3-Large, RegNet-Y-400MF, ConvNeXt-Tiny, ConvNeXt-Small, ViT-B/16 и Swin-T. Линейка кандидатов идет от компактных сверточных схем к ConvNeXt и трансформерным вариантам, чтобы не сужать поиск одним индуктивным смещением и не задавать заранее единый масштаб представления.

Отбор архитектуры для каждого атрибута идет в три прохода. Сначала замеряется качество с предобученным извлекателем и классификационной головой без дообучения. Становится видно, что дают одни представления ImageNet. Потом короткое дообучение каждого кандидата на малом числе эпох и отсев слабых сетей. Оставшиеся обучаются полным циклом. Победитель на валидации фиксируется как итоговая модель для этого атрибута.

Такое обучение дороже, чем заранее зафиксированная архитектура, зато метрики по атрибутам не привязаны к случайному выбору семейства, а этапы по разным атрибутам можно выполнять параллельно. Чтобы фиксированный порядок прогона кандидатов не давал постоянного перевеса одним и тем же семействам сетей, порядок обхода кандидатов меняется от атрибута к атрибуту.

Для всех атрибутов в базовой конфигурации используется функция потерь кросс-энтропии. Конкретные гиперпараметры – скорость обучения, размер пакета, число эпох, регуляризация, оптимизатор и расписание скорости – описаны в следующем разделе.

Главный критерий сравнения и отбора – F1-мера с макроусреднением: классы вносят равный вклад, что уместно при перекосе классов в

подготовленных данных. Дополнительно снимается точность, полнота и доля верных ответов по каждому классу, чтобы видеть слабые категории.

Общая схема четырех независимых моделей изображена на рисунке 11.

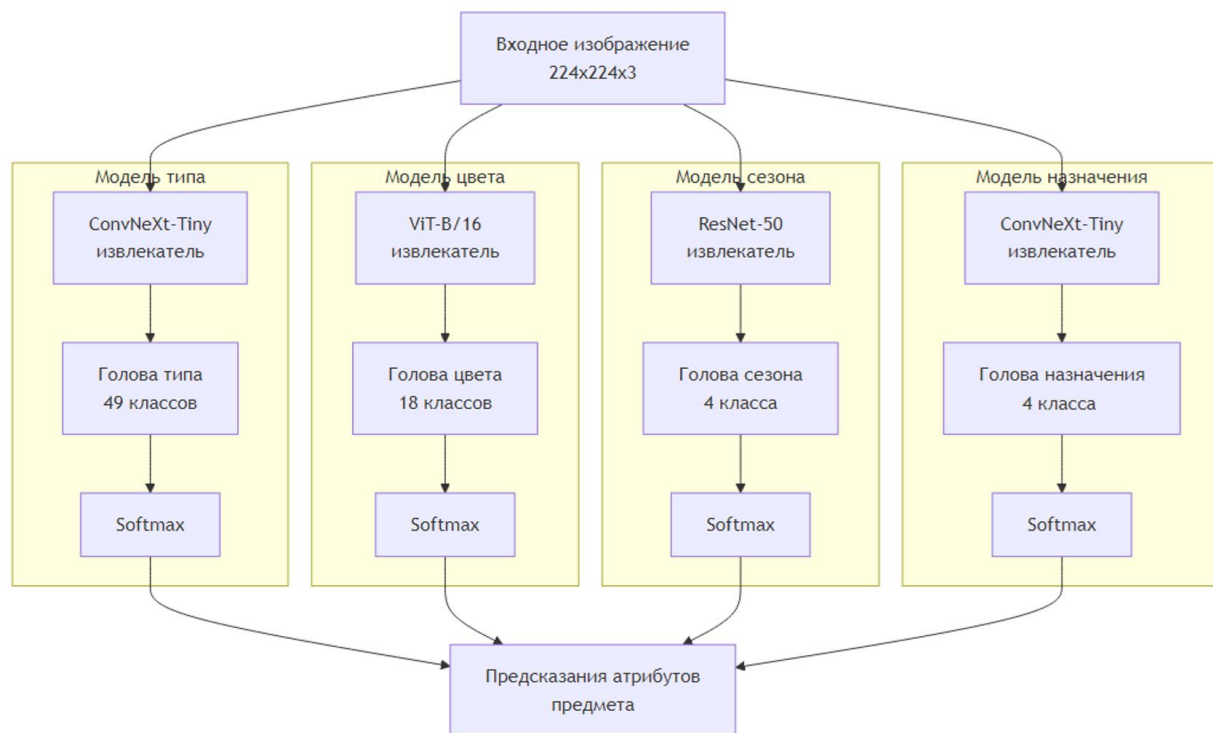


Рисунок 11 – Схема классификации атрибутов одежды. Источник: Составлено автором

3.2 Разработка модели оценки совместимости элементов гардероба

Совместимость комплекта здесь сводится к бинарной задаче: образ «согласован» или «не согласован». На входе три вещи – верх, низ и обувь. На выходе – оценка вероятности согласованности. Такая постановка стыкуется с алгоритмом сборки образа в приложении: ту же модель можно применить и к пользовательскому комплекту, и к кандидатам из рекомендаций.

Обучающий материал – Polyvore Outfits. Из каждого пользовательского комплекта собирается триплет: к верху относятся классы tops и outerwear, к низу – bottoms, к обуви – shoes. Образы без полного триплета отбрасываются. Отрицательные примеры получаются подменой одной или нескольких позиций на случайные вещи той же группы из других образов. Доли классов

получаются близкими к 50/50, так что отдельная балансировка классов почти не нужна, а ассигасу остается читаемой.

Конвейер состоит из двух блоков. Сначала каждая вещь прогоняется через извлекатель признаков без классификационной головы. Основной вариант – извлекатель из моделей атрибутов, уже обученный на типе, цвете, сезоне и назначении: эти сигналы близки к тому, что люди учитывают в сочетаемости. Для сравнения берутся и базовые линии: ResNet-18, ResNet-34, EfficientNet-B0, инициализированные только весами ImageNet без дообучения на атрибутах.

Эмбеддинги трех предметов сцепляются в фиксированном порядке, чтобы роль вещи кодировалась позицией в векторе: верх, низ и обувь. Перед классификатором признаки стандартизуются – вычитается среднее и происходит деление на стандартное отклонение по каждой компоненте, оцененные на обучающей выборке.

Второй блок – не глубокая сеть, а классические методы по уже готовому вектору: логистическая регрессия, MLP (Multi-Layer Perceptron), SVM с RBF-ядром, случайный лес, GradientBoosting из sklearn, LightGBM, XGBoost и CatBoost как представители градиентного бустинга [59]. Подходы с сиамскими сетями и графовыми методами, рассмотренные в первой главе, дают лучшие числа на крупных корпусах, но требуют совместного дообучения извлекателя и модели совместимости, что ломает независимость модулей: при обновлении атрибутивных классификаторов пришлось бы переучивать и совместимость. Объем доступной выборки комплектов умеренный, а эмбеддинги уже достаточно информативны – качество, достигнутое классическими методами, соответствует требованиям прикладного сценария.

Поверх отдельных моделей пробуются мягкое голосование (VotingClassifier, soft voting): усредняются вероятности нескольких сильных и разнообразных базовых алгоритмов, чтобы сгладить разброс ответов. Состав и веса подбираются по валидации. Веса связываются с качеством каждой базовой модели на валидационной подвыборке.

Разнесение эмбединга и классификатора упрощает сопровождение: обновили атрибутные модели – обновился и источник векторов для совместимости без полного переобучения всего контура. Фиксированный триплет совпадает с тем, как в приложении собирается образ из гардероба. Легкие модели по конкатенированному вектору дают более короткий ответ на сервере, чем монолитная глубокая сеть на весь комплект.

Основной ориентир при сравнении конфигураций – ассигасу (классы почти сбалансированы). Рядом просматриваются ROC-AUC, F1, точность и полнота. Цифры приведены в сводных таблицах этой главы.

Схема цепочки обработки представлена на рисунке 12.

3.3 Обучение моделей и подбор гиперпараметров

Итоговое качество заметно зависит от скорости обучения, размера пакета, числа эпох, регуляризации и вида расписания скорости.

Классификаторы атрибутов обучаются в два этапа. Сначала каждая кандидатная архитектура прогоняется всего две эпохи в коротком цикле предварительного отбора, после чего по F1-мере с макроусреднением на валидации отсеиваются слабые сети. Победитель по каждому атрибуту доучивается полным циклом до сходимости.

В качестве оптимизатора – AdamW [60], расписание – косинусное затухание скорости на заданное число эпох. Сид случайности для обучения – 42, чтобы повторный запуск на тех же данных давал ту же траекторию аугментаций и инициализации.

Основной прогон задается параметрами в конфигурации обучения: 28 эпох, пакет 48, вход 224×224 , коэффициент отдельного штрафа на веса в AdamW 0,0001. Стартовая скорость обучения подобрана отдельно для каждого атрибута в коридоре 0,0002-0,00022, так как разные извлекатели признаков по-разному реагируют на оптимизатор. После отбора были закреплены ConvNeXt-Tiny для типа предмета и назначения, ViT-B/16 для базового цвета, ResNet-50 для сезона.

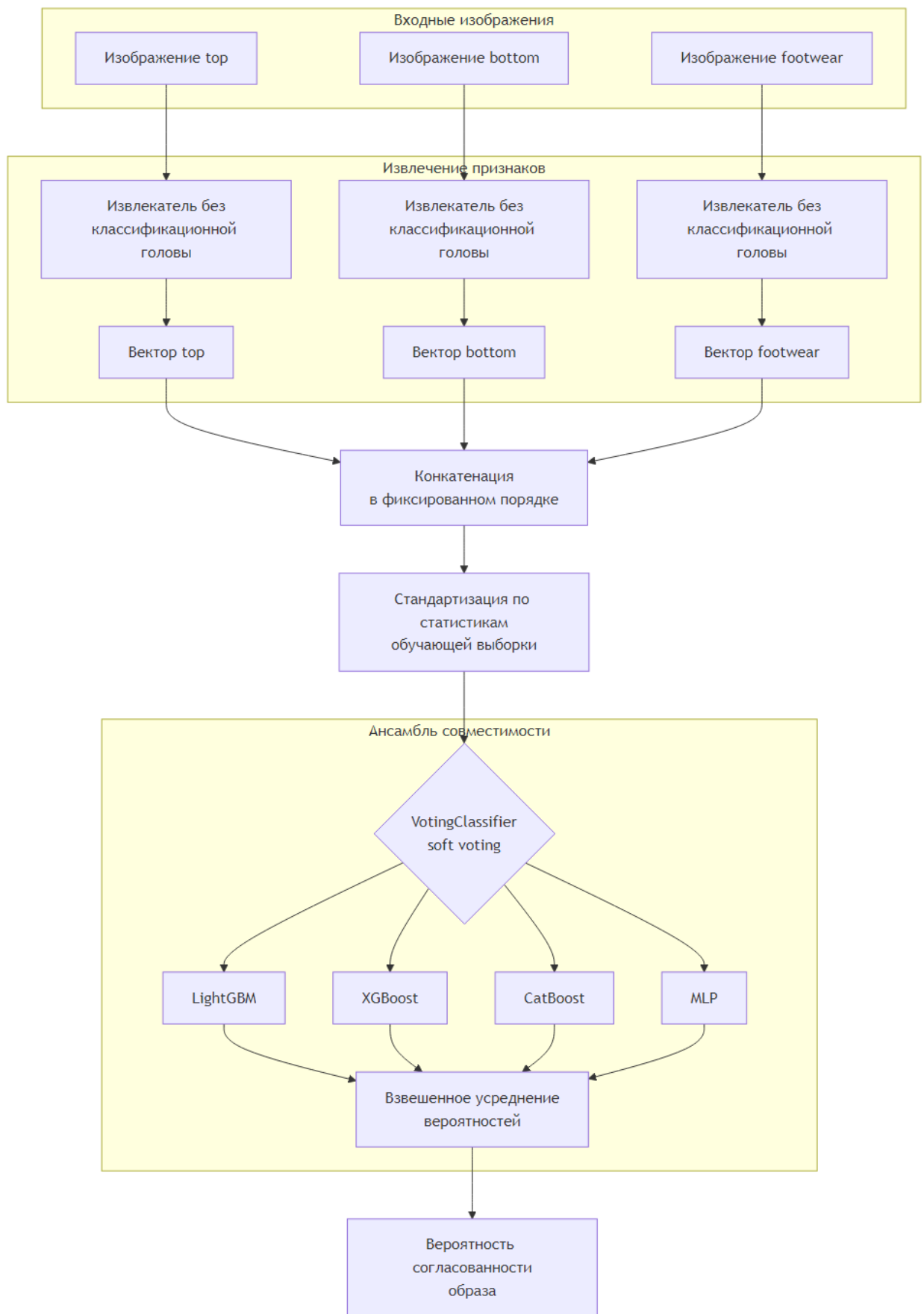


Рисунок 12 – Схема цепочки обработки оценки совместимости комплекта.
Источник: Составлено автором

Также используется ранняя остановка: если пять эпох подряд прирост целевой метрики на валидации не превышает 0,0005, обучение останавливается. Лучшие по валидации веса сохраняются отдельно, чтобы итоговая модель не обязательно совпадала с последней эпохой, когда метрика уже могла упасть.

Для модели совместимости гиперпараметры подбираются по отдельности для каждого классического классификатора из контура оценки совместимости выше. Происходит перебор по сетке с кросс-валидацией внутри обучающей части. В сетку входят типичные настраиваемые параметры: для логистической регрессии – C . Для SVM – C и γ . Для случайного леса – число деревьев, глубина, минимальный размер листа. Для бустинга – число итераций, скорость обучения, глубина деревьев, $L1/L2$. Для MLP – конфигурация скрытых слоев, скорость обучения, α . Отдельно перебирается число компонент PCA от 64 до полной размерности конкатенированного эмбединга.

Ансамбль с мягким голосованием собирается из трех-четырех моделей с высоким качеством на валидации и по возможности разным устройством решения, чтобы ошибки меньше совпадали и модель работала лучше. Веса в голосовании ставятся пропорционально доле верных ответов (ассигура) на валидации.

Разбиение на обучающую, валидационную и тестовую части повторяет пропорции 70/15/15 и правила стратификации, принятые для атрибутивных моделей. Подбор архитектур и гиперпараметров опирается только на валидацию. Тестовая часть используется только один раз для итоговых метрик.

Для воспроизводимости результатов фиксируются сиды в стандартном модуле random, NumPy и PyTorch, сохраняются конфиги, веса моделей PyTorch и логи метрик по эпохам.

3.4 Сравнительный анализ моделей по выбранным метрикам

Здесь приведены цифры по двум линиям: четыре классификатора атрибутов и оценка совместимости комплекта. Таблица 4 фиксирует F1 с макроусреднением на валидации в ходе отбора извлекателя признаков – так сравниваются одиннадцать кандидатов. Метрики совместимости – в таблице 5, на тестовой части Polyvore.

Таблица 4 – Значения F1-меры с макроусреднением на валидационной подвыборке для рассмотренных архитектур извлекателей признаков. Источник: Составлено автором

Архитектура извлекателя признаков	Голова type	Голова color	Голова season	Голова usage
1	2	3	4	5
ResNet-18	0,842	0,668	0,572	0,849
ResNet-34	0,857	0,681	0,586	0,861
ResNet-50	0,873	0,694	0,617	0,872
EfficientNet-B0	0,864	0,684	0,594	0,866
EfficientNet-B1	0,871	0,690	0,602	0,874
MobileNetV3- Large	0,848	0,672	0,580	0,853
RegNet-Y- 400MF	0,852	0,676	0,585	0,857
ConvNeXt-Tiny	0,899	0,703	0,611	0,891
ConvNeXt-Small	0,894	0,706	0,607	0,887
ViT-B/16	0,886	0,715	0,604	0,882

Продолжение таблицы 4

1	2	3	4	5
Swin-T	0,881	0,709	0,599	0,878
Итоговый выбор	ConvNeXt-Tiny	ViT-B/16	ResNet-50	ConvNeXt-Tiny

По строке «итоговый выбор» видно, что тип и назначение лучше всего определяются на ConvNeXt-Tiny (0,899 и 0,891 по F1-мере с макроусреднением на валидации), цвет – на ViT-B/16 (0,715), сезон – на ResNet-50 (0,617). ConvNeXt-Small близок к Tiny по ряду столбцов, но итог зафиксирован как в нижней строке таблицы.

Разброс по атрибутам ожидаем. Тип и назначение хорошо читаются по силуэту и деталям кроя на каталожных снимках сверху: разница между брюками, юбкой и платьем устойчиво видна в пикселях. Цвет слабее из-за многоцветных вещей, спорных оттенков и неодинакового освещения. Сезон – наиболее трудный атрибут: одна вещь может носиться в разные сезоны и получить ту или иную метку по маркетинговому решению производителя, а не по жесткому визуальному признаку. Этим объясняется и то, что лучший результат по сезону (ResNet-50, $F1 = 0,617$) заметно уступает остальным атрибутам при полном сравнительном цикле – потолок определяется шумом в разметке, а не слабостью архитектуры. Порядок величин близок тому, что обычно получают на Kaggle Fashion Product Images при похожей разметке.

Нормированные матрицы ошибок на рисунках 13-16 показывают типичные путаницы на тестовой выборке. По типу чаще путают близкие силуэты: рубашки и блузы, кардиганы и свитеры, близкие типы обуви. По цвету страдают бежевый/коричневый, фиолетовый/синий и вещи, где в разметке смешаны несколько оттенков.

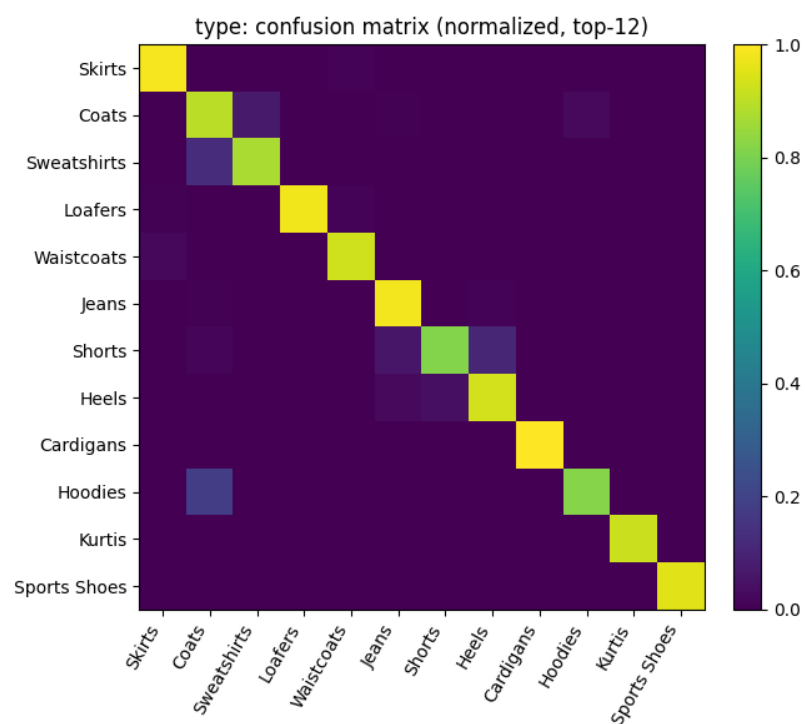


Рисунок 13 – Нормированная матрица несоответствий для головы классификации типа предмета одежды на тестовой подвыборке. Источник: Составлено автором

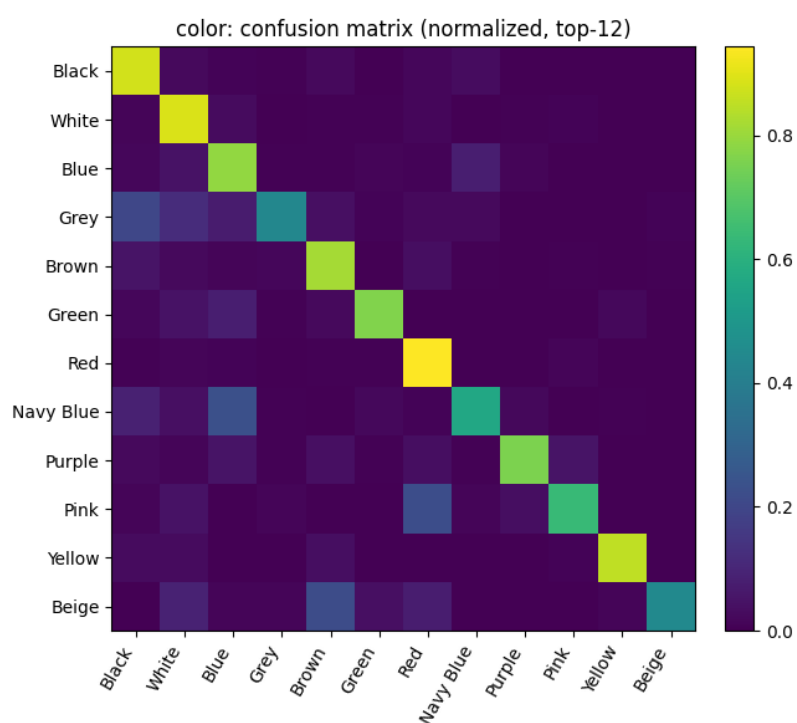


Рисунок 14 – Нормированная матрица несоответствий для головы классификации базового цвета на тестовой подвыборке. Источник: Составлено автором

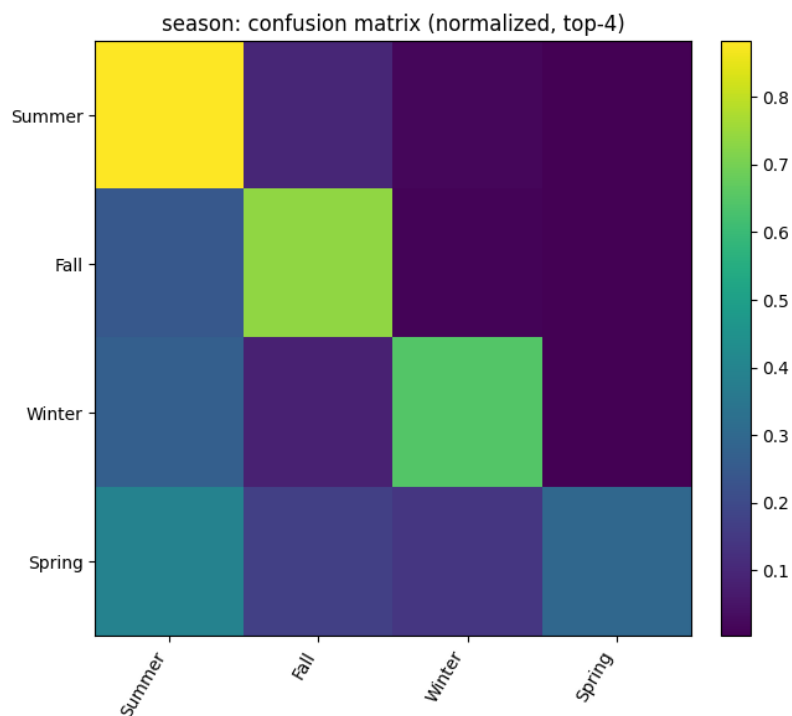


Рисунок 15 – Нормированная матрица несоответствий для головы классификации сезонной принадлежности на тестовой подвыборке.
Источник: Составлено автором

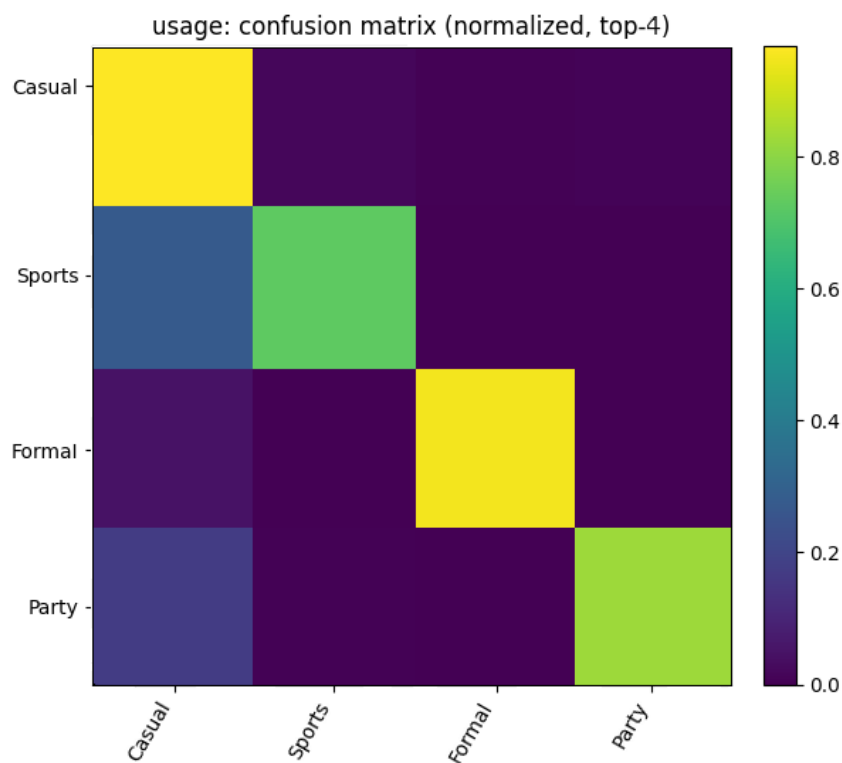


Рисунок 16 – Нормированная матрица несоответствий для головы классификации назначения на тестовой подвыборке. Источник: Составлено автором

Примеры предсказания типа одежды представлены на рисунке 17.

GT: Casual Shoes
Pred: Casual Shoes



GT: Sports Shoes
Pred: Sports Shoes



GT: Tops
Pred: Tops



GT: Tshirts
Pred: Tshirts



Рисунок 17 – Примеры предсказаний модели классификации типа одежды на тестовой выборке. Источник: Составлено автором

Тест Polyvore для бинарной совместимости – в таблице 5. Признаки для всех строк строятся тем же извлекателем признаков, что и у классификатора типа предмета.

Таблица 5 – Значения метрик качества моделей оценки совместимости элементов гардероба на тестовой подвыборке набора Polyvore Outfits. Источник: Составлено автором

Алгоритм классификации	Ассигасу	F1-мера	ROC-AUC	Среднее время вывода на один образ, мс
1	2	3	4	5
Логистическая регрессия	0,724	0,718	0,784	0,4
Метод опорных векторов (SVM)	0,742	0,735	0,812	1,2

1	2	3	4	5
Многослойный перцептрон (MLP)	0,763	0,758	0,832	0,8
Случайный лес	0,771	0,764	0,843	1,5
Градиентный бустинг	0,821	0,815	0,896	2,0
LightGBM	0,842	0,835	0,914	0,8
XGBoost	0,840	0,833	0,912	1,0
CatBoost	0,849	0,842	0,919	1,5
VotingClassifier (мягкое голосование)	0,861	0,854	0,928	3,0

Линейные модели (логистическая регрессия, SVM) отстают от бустинга: у LightGBM, XGBoost и CatBoost выше и accuracy, и ROC-AUC. Мягкое голосование чуть поднимает все три метрики до лучших в таблице.

ROC-AUC и F1 идут в одном ранге с accuracy – классы почти сбалансированы, сильного компромисса точность/полнота между методами не видно. VotingClassifier остается разумным компромиссом качества и задержки для сервера.

В итоговой конфигурации системы оставляют: ConvNeXt-Tiny для типа, ViT-B/16 для цвета, ResNet-50 для сезона, ConvNeXt-Tiny для назначения. Для совместимости – VotingClassifier над сильными базами (бустинг и MLP) на эмбедингах из модели типа предмета.

3.5 Разработка алгоритма формирования образов

Приложение должно собирать из гардероба пользователя варианты комплектов, оценивать их моделью совместимости и отдавать отсортированный список с числом баллов по каждому. Атрибутные модели отрабатывают сразу при добавлении новой одежды: в гардеробе лежат id вещи, ее категория и сохраненный эмбединг из соответствующего извлекателя признаков.

На входе в модель совместимости – множество предметов с идентификаторами, категориями и векторами признаков. На выходе – не больше K образов (по умолчанию 10) с убыванием оценки. У каждого образа – три id (или два, если одной роли нет) и сама оценка, чтобы при необходимости отфильтровать слабые рекомендации или показать степень уверенности.

Сначала вещи делятся на три роли по той же схеме триплета, что и в задаче совместимости: верх, низ и обувь. Одна роль – один слот в комплекте.

Комбинации строятся перебором: не больше одной вещи из каждой занятой роли. Если все три роли не пусты, теоретически перебираются все тройки с одной вещью на роль верх, низ и обувь. Если одной роли нет, остаются пары из двух оставшихся.

Полный перебор на больших гардеробах дорогой, поэтому действуют два ограничения. Из каждой роли берется не больше заданной константы `max_per_role`, равной по умолчанию 25 элементам (на практике для повседневного подбора этого обычно хватает). Число просмотренных комбинаций ограничивается константой `max_combinations`, равной по умолчанию 5000, чтобы ответ укладывался в разумное время при любом входе.

Для каждой комбинации собирается признаковый вектор для `VotingClassifier` из трех кусков. Первый – среднее эмбедингов всех вещей в комплекте: дает общий «центр» образа в пространстве признаков и отражает стилевой регион, к которому тяготеет весь набор. Второй – агрегаты по попарным косинусным близостям и евклидовым расстояниям: минимум,

среднее, максимум, стандартное отклонение и число предметов; эти статистики кодируют внутреннюю однородность комплекта и выявляют выбивающиеся предметы, которые среднее скрывает. Третий – нормированное распределение категорий внутри комплекта: позволяет классификатору разграничить полный образ (верх, низ и обувь) и неполный (пара предметов), а также учесть нетипичные сочетания ролей. Предварительные прогоны показали, что исключение любого из трех блоков снижает ассигасу на валидации: каждый компонент вносит информацию, которую остальные не перекрывают.

Пример работы модели совместимости элементов одежды представлен на рисунке 18.



Рисунок 18 – Пример предсказания обученной модели в задаче оценки совместимости на тестовой выборке. Источник: Составлено автором

Все оценки сортируются по убыванию, после чего возвращается К лучших. Список идет в клиентский интерфейс и в остальную логику рекомендаций.

Эмбединги считаются при добавлении вещи в гардероб и хранятся вместе с записью, поэтому при каждом запросе рекомендаций тяжелый извлекатель признаков повторно не прогоняется. Верхний классификатор совместимости дает короткую оценку на одну комбинацию. Время ответа на сервере остается приемлемым.

Блок-схема алгоритма представлена на рисунке 19.

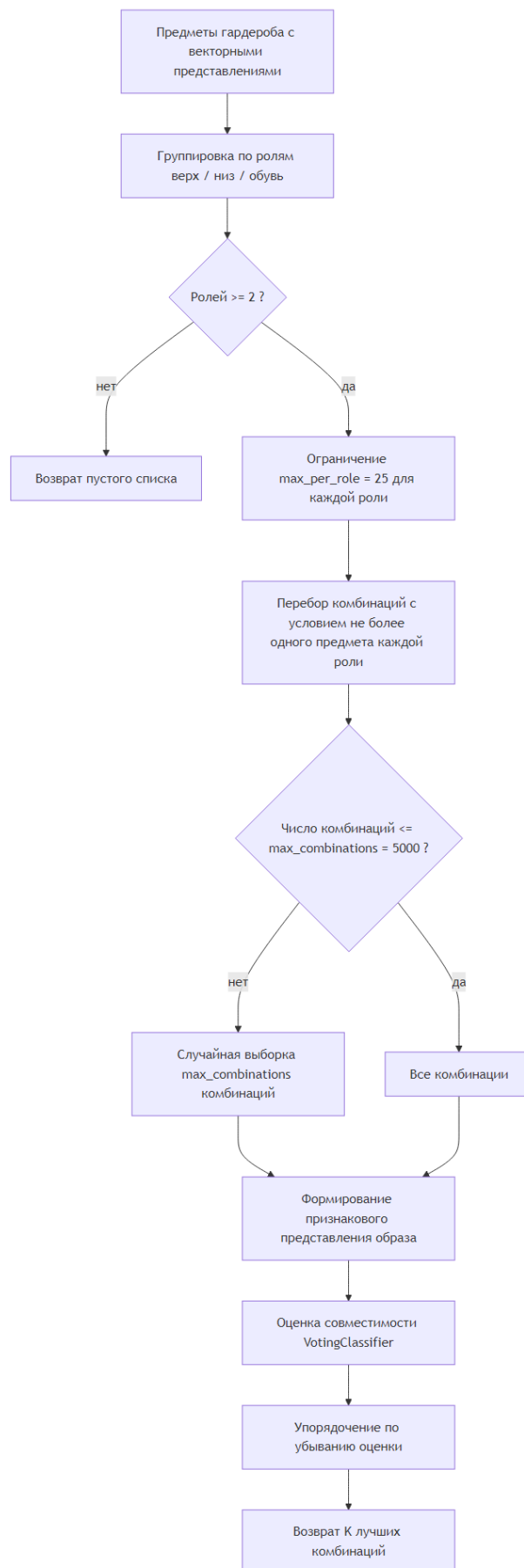


Рисунок 19 – Блок-схема алгоритма формирования образов. Источник:
Составлено автором

Фрагмент кода с группировкой по ролям, обрезкой до `max_per_role` и перебором до `max_combinations` – в листинге 2.

Листинг 2 – Группировка предметов по ролям и формирование комбинаций в составе алгоритма формирования образов. Источник: Составлено автором

```
role_cats = {"top": ["tops", "outerwear"], "bottom": ["bottoms"],
"shoes": ["shoes"]}
by_role: dict[str, list[tuple[str, np.ndarray, str]]] = {
    "top": [], "bottom": [], "shoes": []
}
for item_id, emb, cat in items:
    emb_arr = np.array(emb, dtype=np.float32)
    for role, cats in role_cats.items():
        if cat in cats:
            by_role[role].append((item_id, emb_arr, cat))
            break

max_per_role = 25
for role in by_role:
    by_role[role] = by_role[role][:max_per_role]

tops, bottoms, shoes = by_role["top"], by_role["bottom"],
by_role["shoes"]
scored: list[tuple[list[str], float]] = []
count = 0

if tops and bottoms and shoes:
    for (id_t, e_t, c_t) in tops:
        for (id_b, e_b, c_b) in bottoms:
            for (id_s, e_s, c_s) in shoes:
                if count >= max_combinations:
                    break
                add_combos([(id_t, e_t, c_t), (id_b, e_b, c_b),
(id_s, e_s, c_s)])
                if count >= max_combinations:
                    break
            if count >= max_combinations:
                break

scored.sort(key=lambda x: x[1], reverse=True)
return scored[:top_k]
```

Дозаполнение неполного комплекта (когда выбраны не все слоты) вынесено в отдельный сценарий: здесь отбираются кандидаты по сочетанию визуальной близости и совместимости к исходным вещам. Пример рекомендации недостающей вещи представлен на рисунке 20.



Рисунок 20 – Пример рекомендации недостающей вещи. Источник: Составлено автором

3.6 Выводы по третьей главе

Третья глава сводит воедино обученные модели и логику рекомендаций: четыре классификатора атрибутов, оценка совместимости комплекта и переборно-ранжирующий алгоритм образов. Для каждой линии зафиксированы архитектура, схема обучения и цифры качества в сводных таблицах этой главы.

Атрибуты выявляются отдельными сетями (отдельная модель на каждый атрибут): так проще подобрать извлекатель под разное число классов и разный визуальный сигнал, независимо обновлять веса и не упираться в общий компромисс одной мультиголовой модели. Из одиннадцати кандидатов на извлекатель признаков отобран ConvNeXt-Tiny для типа и назначения, ViT-B/16 для базового цвета, ResNet-50 для сезона.

На тестовой выборке итоговые значения F1-меры с макроусреднением по атрибутам: 0,899 (тип), 0,891 (назначение), 0,715 (цвет), 0,617 (сезон). Разброс отражает и силу визуального паттерна, и шум в разметке Kaggle-набора. Для прикладного гардероба этого набора метрик достаточно.

Совместимость строится на эмбедингах из атрибутного извлекателя и классическом классификаторе по вектору признаков образа: среднее по предметам, статистики попарных близостей и нормированное распределение категорий. Среди логистической регрессии, MLP, SVM, случайного леса и

бустингов лучше всего показал себя ансамбль с мягким голосованием сильных базовых моделей.

Готовый алгоритм построения образов перебирает допустимые комплекты из гардероба (не больше одной вещи на роль), считает оценку совместимости, сортирует и отдает К лучших.

Набор из четырех атрибутивных классификаторов, ансамблевого оценщика совместимости и алгоритма перебора образов покрывает весь ML-контур системы – третья глава закрыла экспериментальную часть работы. Дальше – встраивание этого набора в веб-систему: архитектура, сервис запросов и связка с приложением раскрываются в следующей главе.

4 ИНТЕГРАЦИЯ МОДУЛЯ МАШИННОГО ОБУЧЕНИЯ В ВЕБ-ПРИЛОЖЕНИЕ

Глава показывает, как обученные модели встраиваются в рабочую систему. Первый раздел описывает архитектуру: ML-сервис выделен в отдельный Docker-контейнер и взаимодействует с основным бэкендом по внутреннему HTTP. Следующий разбирает реализацию сервиса на FastAPI – схемы запросов и ответов, маршруты, управление моделями в памяти. Далее охвачена интеграция со стороны основного приложения: как добавление вещи, атрибутивный поиск и подбор комплектов вызывают ML-сервис, и описано тестирование системы в сборке. Веб-приложение включает привычный контур – интерфейс, сервер приложения, реляционную базу и объектное хранилище – и ML-сервис подключается к нему как отдельный компонент.

4.1 Архитектура системы

Для ML-сервиса реализация выполнена на Python с FastAPI и PyTorch. Сервис вызывается только из внутренней сети и не хранит состояние между запросами: в теле запроса передают все данные, нужные для вывода модели. Так проще масштабировать репликами за балансировщиком и обновлять веса без простоя остальной системы.

Добавление вещи в прикладном сценарии опирается на три последовательных вызова моделей (фон, атрибуты, вектор), чтобы показать промежуточные результаты пользователю и дать поправить метки до сохранения. После подтверждения атрибуты и эмбединг сохраняются в хранилищах приложения, и дальнейший подбор комплектов читает уже сохраненный вектор без повторного тяжелого прогона извлекавателя.

ML-сервис упакован в отдельный Docker-образ с фиксированными версиями CUDA, PyTorch и библиотек сегментации. Веса атрибутивных моделей, модель совместимости, параметры нормализации признаков и таблицы соответствия индексов классов строкам меток монтируются томом.

При старте они загружаются в память. Для разработки используется docker-compose, для эксплуатации – типичный контур Kubernetes.

Общая схема взаимодействия – на рисунке 21.

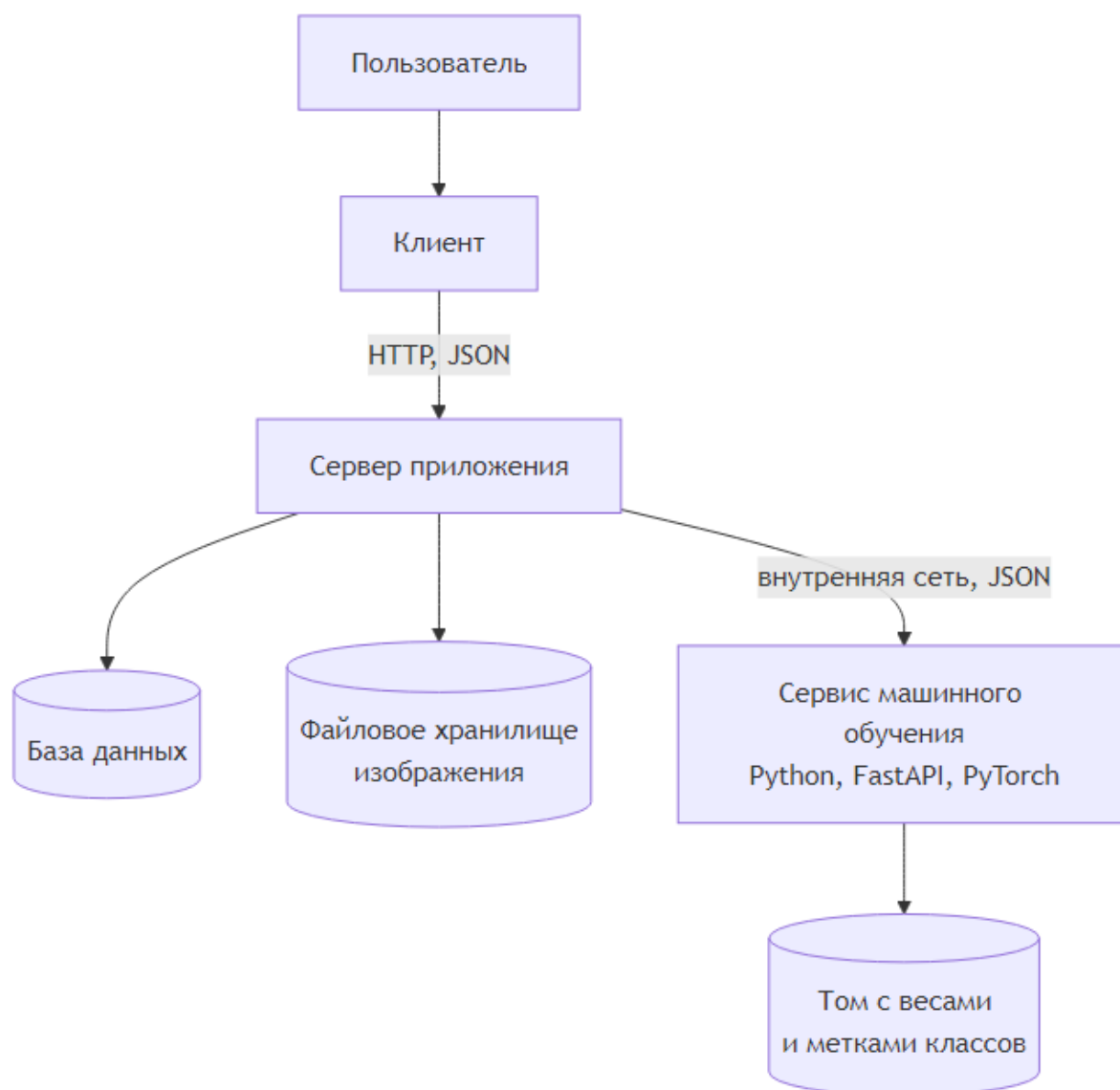


Рисунок 21 – Схема архитектуры разработанной системы. Источник: Составлено автором

4.2 Разработка сервиса обработки запросов

ML-модуль в системе – отдельное серверное приложение: по внутреннему HTTP (HyperText Transfer Protocol) оно исполняет модели третьей главы.

Опорный набор технологий – FastAPI на Python. ASGI и uvicorn дают привычную для веб-сервиса модель параллельной обработки соединений.

Вход описывается типизированными схемами тел запросов и ответов (реализованы средствами Pydantic): несовпадение со схемой превращается в ответ с ошибкой. По аннотированным сигнатурам обработчиков строится OpenAPI – удобно для Swagger [61] и для генерации клиента у вызывающей стороны. PyTorch остается тем же, что при обучении, без экспорта в другую среду выполнения, чтобы не расходились результаты на этапе обучения и в эксплуатации.

Код разнесен по слоям. На уровне точки входа поднимается экземпляр FastAPI, подключаются CORS (Cross-Origin Resource Sharing), маршруты и обработчик жизненного цикла. Слой HTTP сосредоточивает прием multipart и JSON, вызовы прикладной логики и выбор кода ответа. Прикладной слой держит предметную логику: удаление фона, атрибуты, эмбединги, а также операции с образом – оценка совместимости, сборка комплекта, рекомендация по незаполненному слоту, поиск визуально близких готовых образов. Конфигурация задает пути к весам, параметры предобработки изображений и прочие величины, считываемые при разворачивании. Схема слоев сервиса обработки запросов представлена на рисунке 22.

Настройки берутся из переменных окружения при загрузке конфигурации: путь к каталогу с весами и сопутствующими файлами, при необходимости отдельные пути к сериализованным классификаторам, а также параметры коррекции баланса белого.

После прихода запроса FastAPI разбирает тело: файл уходит в буфер байтов, JSON – в типизированный объект по схеме, заданной в коде. Обработчик сам отсекает пустые файлы и недопустимые параметры и отвечает ошибкой с пояснением. Далее вызывается прикладной слой: предобработка, прогон моделей, сбор результата. Если на диске нет нужных сериализованных весов, сервисный слой сигнализирует об этом – обработчик отвечает ошибкой сервера с пояснением, каких артефактов не хватает в каталоге моделей. При успехе отдается успешный статус и JSON. Исключение – `remove-bg`, который отдает бинарный PNG.

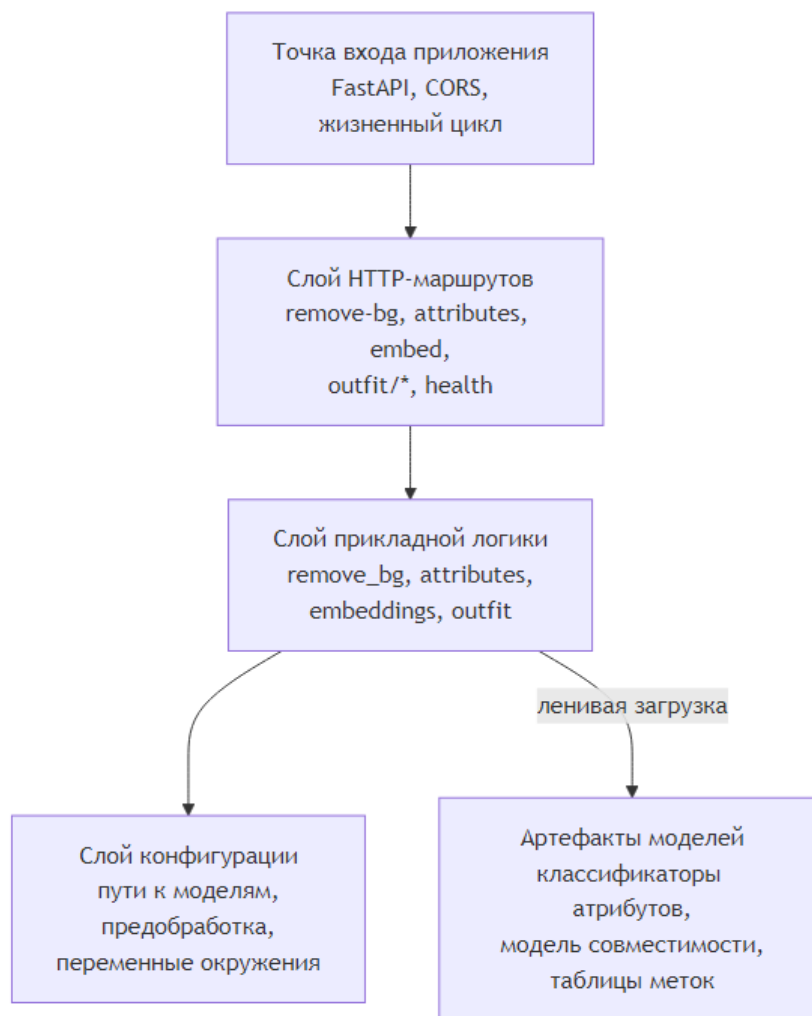


Рисунок 22 – Схема слоев сервиса обработки запросов и их взаимодействия.
Источник: Составлено автором

Веса подгружаются лениво: при старте процесса они не читаются в память. Первый вызов соответствующего блока загружает файл. Объект хранится в кэше модуля и дальше переиспользуется без повторного чтения с диска. Холодный старт короче, неиспользуемая в данном экземпляре ветка не резервирует память. После прогрева задержка близка к чистому применению модели без накладных расходов на загрузку.

Маршруты делятся на работу с изображением предмета (удаление фона, атрибуты, эмбеддинг) и на операции над векторами в пространстве признаков (оценка, сборка и рекомендации образа, поиск похожих). Ниже рассмотрены некоторые маршруты.

POST `/api/ml/remove-bg` принимает `multipart` с изображением и возвращает PNG: альфа задает маску переднего плана. Внутри – `rembg`, наложение маски на исходное изображение, обрезка по непрозрачной области.

`/api/ml/attributes` принимает изображение предмета и отдает JSON: тип, базовый цвет, сезон и назначение (в API (Application Programming Interface) те же поля, что и для остальных атрибутов), а также оценки уверенности по каждому из возвращаемых атрибутов в диапазоне от 0 до 1. Уверенность – максимум по выходу `softmax` соответствующей головы. Здесь четыре классификатора вызываются в одной цепочке обработки изображения.

`/api/ml/embed` принимает файл и параметр размерности эмбединга. Далее вектор строится извлекателем на базе `ConvNeXt-Tiny` и дальше используется в `outfit/score` и `outfit/build`.

`/api/ml/outfit/score` принимает эмбединги предметов образа и их роли (верх, низ и обувь), возвращает скаляр совместимости в диапазоне от 0 до 1. Вектор признаков образа собирается по правилам алгоритма сборки комплекта, классификатор берется из обученного ансамбля на эмбедингах.

`/api/ml/outfit/build` получает гардероб (`id` предметов, векторы, категории). В ответе – список образов с убывающей оценкой совместимости и набором `id` составляющих. Реализован тот же переборно-ранжирующий алгоритм, что в описании сборки образа, с ограничениями на число предметов каждой роли и на число перебираемых комбинаций, чтобы вернуть сильные по модели наборы из имеющихся вещей.

`/api/ml/outfit/recommend` обслуживает частично собранный образ: уже выбранные предметы, целевой слот и набор кандидатов. Ранжирование сочетает визуальную близость и согласованность кандидата к выбранному и оценку полного образа после добавления кандидата.

`/api/ml/outfit/similar` – маршрут, подходящий для поиска визуально близких готовых образов и смежных рекомендательным сценариям. Он принимает вектор образа и векторы кандидатов. Порядок ответа – по убыванию косинусной близости.

/api/health отдает JSON о том, что процесс жив и какие группы весов найдены на диске. Его можно использовать в readiness/liveness в Kubernetes.

Документация построена из типов полей и текстовых описаний обработчиков в исходном коде. Интерактивная документация – /docs (Swagger UI), читаемый вид – /redoc, машиночитаемый контракт – /openapi.json.

Интерактивная документация представлена на рисунке 23.

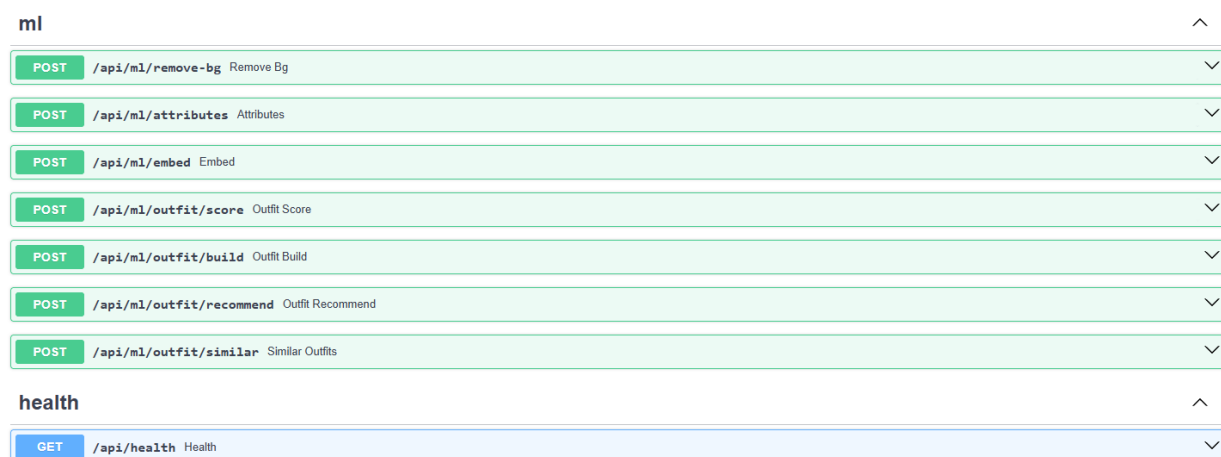


Рисунок 23 – Интерактивная документация REST API сервиса в Swagger UI.
Составлено автором на основе источников [61]

Сервис организован как FastAPI-приложение со слоями, ленивой подгрузкой весов и явным REST-контрактом. Маршруты закрывают классификацию, эмбединги, оценку и сборку образов по логике экспериментального контура.

4.3 Интеграция модели машинного обучения в веб-приложение

Чтобы обученные модели работали в среде эксплуатации, нужно доставить веса и метки в контейнер ML-сервиса, поднять их тем же кодом, что и при обучении, и согласовать вызовы с остальным приложением: добавление вещи, сборка образа, дозаполнение слота, поиск похожих комплектов.

Классификаторы атрибутов сериализуются средствами PyTorch: в одном артефакте хранятся численные параметры сети и обозначение выбранной архитектуры, чтобы при развертывании автоматически восстановить тот же каркас, что и при обучении, без жесткой привязки к одному имени класса в

исходном коде. Модель совместимости сохраняется через `joblib` вместе с обученным ансамблем классических классификаторов и параметрами нормализации признаков.

Рядом с весами хранится таблица меток: для каждого атрибута – упорядоченный список строк классов, чтобы индекс с выхода головы переводить в ответ API. Размеры списков те же, что при обучении: тип – 49 значений, цвет – 18, сезон – 4, назначение – 4. Общий файл меток с весами фиксирует согласованность пространств классов между обучением и применением моделей.

Далее том монтируется в каталог, предусмотренный конфигурацией контейнера, без зашивания весов в образ. Обновление – замена файлов и рестарт пода, без обязательной пересборки. Если бы файлы весов лежали только внутри образа, каждое обновление модели тянуло бы за собой пересборку и лишний риск ошибки в `Dockerfile`. Путь к каталогу задается при развертывании и его можно менять между стендом и эксплуатацией.

Загрузка атрибутного классификатора из файла сводится к восстановлению сохраненных весов и идентификатора архитектуры. По нему поднимается базовая сеть с предобучением ImageNet, заменяется выходной слой на классификатор с числом выходов по таблице меток и подставляются сохраненные параметры. Перед обработкой запросов модель переводится в режим вывода: `dropout` отключен, пакетная нормализация опирается на накопленные при обучении статистики каналов, а не переоценивает их по текущей подвыборке в пакете. При наличии GPU (Graphics Processing Unit) вычисления выполняются на графическом ускорителе.

Предобработка перед головами атрибутов совпадает с валидацией при обучении. Pillow открывает байты из запроса и выставляет ориентацию по EXIF (Exchangeable Image File Format), иначе снимки с телефона могут прийти с неверной ориентацией кадра. Удаляется фон, затем размер приводится к 224×224 билинейной интерполяцией, пиксели масштабируются в диапазоне от

0 до 1 и происходит нормализация по среднему и стандартному отклонению каналов ImageNet, согласованная с предобучением извлекателя.

Сервис имеет отдельный извлекатель с головой на каждый атрибут. К одному кадру применяются основные сети (тип, цвет, сезон и назначение). Чтобы не умножать задержку последовательными прогонами, на GPU все головы обрабатывают параллельно (отдельные потоки команд устройства), на CPU (Central Processing Unit) – пулом из нескольких потоков исполнения. Выигрыш по времени близок к линейному от числа параллельно исполняемых голов при достаточных ресурсах.

Таким образом, добавление вещи в гардероб в типовом сценарии выглядит так: пользователь передает фото через интерфейс на сервер приложения, тот последовательно вызывает `/api/ml/remove-bg`, `/api/ml/attributes` и `/api/ml/embed`, показывает атрибуты и превью, после подтверждения сохраняет файлы в объектном хранилище и запись с эмбедингом в базе данных.

Диаграмма обмена для этого сценария изображена на рисунке 24.

В базе данных для предмета сохраняется массив чисел – выход ConvNeXt-Tiny из `/api/ml/embed`. Повторно вызывать извлекатель на каждом запросе outfit не нужно: score и build читают сохраненные векторы, что ускоряет ответ при больших гардеробах.

Сборка комплектов из уже загруженных вещей идет через `/api/ml/outfit/build`: сервер приложения передает в ML идентификаторы, эмбединги и роли предметов вместе с числом возвращаемых вариантов, получает ранжированный список наборов и отдает его в интерфейс.

Дозаполнение слота обслуживает `/api/ml/outfit/recommend`: по уже выбранным вещам и набору кандидатов сервер собирает эмбединги, вызывает ML и возвращает упорядоченный список идентификаторов.

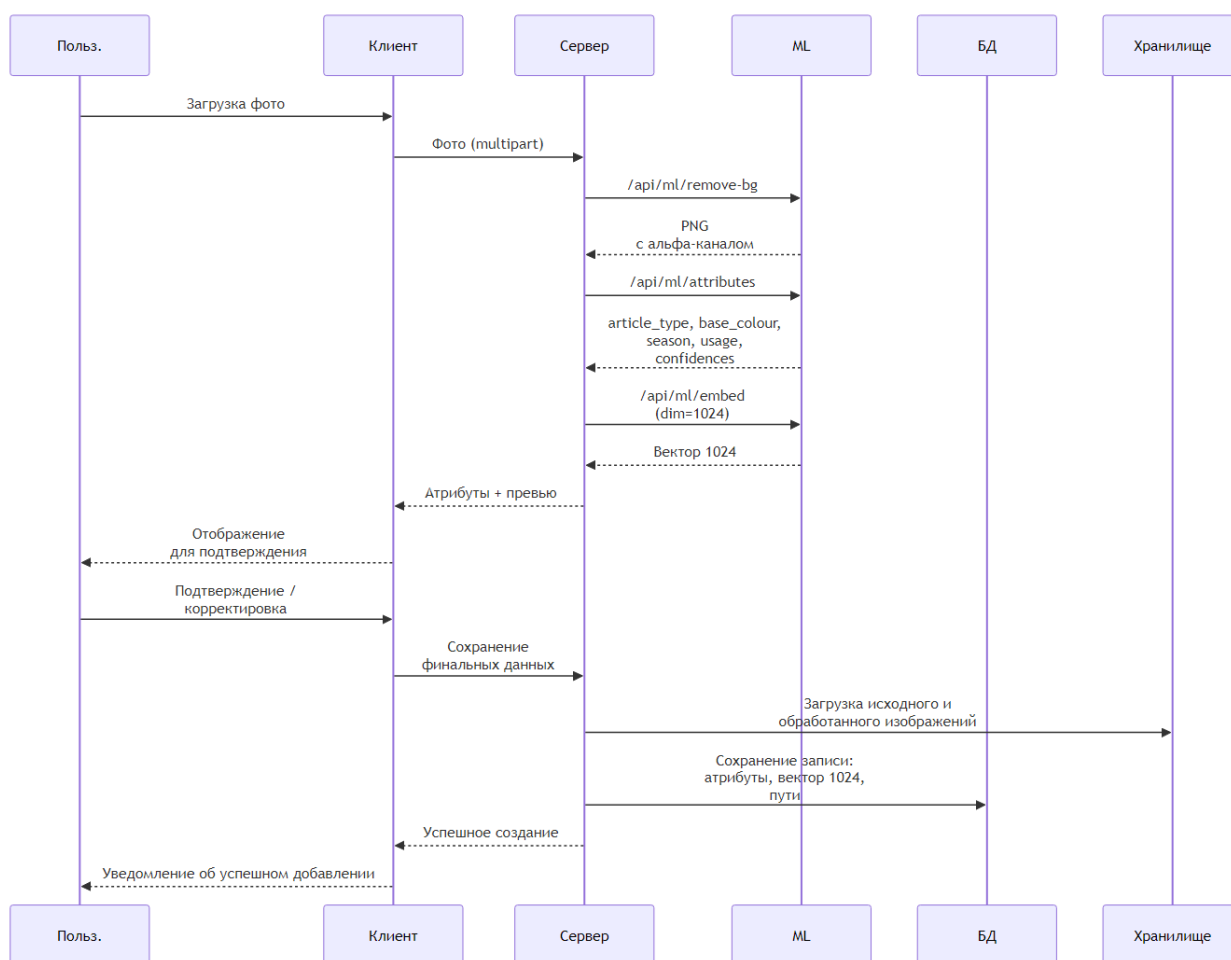


Рисунок 24 – Диаграмма последовательности при добавлении предмета в гардероб. Источник: Составлено автором

Поиск визуально близких образов – `/api/ml/outfit/similar`: по вектору исходного образа и векторам кандидатов ML возвращает упорядочивание по косинусной близости.

Коды ответов ML на стороне приложения переводятся в сообщения пользователю или в журнал. Готовность экземпляров контролируется опросом `/api/health`. По рядам уверенности голов можно отслеживать дрейф качества.

4.4 Тестирование работы системы

Проверки выполнялись на нескольких уровнях: автотесты модулей ML-сервиса на `pytest`, прогон связки сервисов в `docker-compose` в составе, близком к целевой схеме развертывания и ручной прогон пользовательских сценариев. Ниже – состав проверок и выводы по ним.

Модульные тесты написаны на `pytest`. Общие ресурсы вынесены в фикстуры – HTTP-клиент к приложению без поднятия сокета и мелкие JPEG/PNG для `multipart`. Четыре группы тестов соответствуют четырем блокам кода: `remove-bg`, извлечение векторов, логика совместимости, сами HTTP-маршруты.

Для `remove-bg` проверялись PNG с альфой, ужатие большого кадра 2200×1600 до длинной стороны не более 1024 px и обрезка поля прозрачности на синтетическом изображении 200×200 с пятном 80×90 по центру: итоговый кадр меньше исходного, но шире объекта с учетом поля.

Для модуля векторных представлений проверялось, что размерность вектора совпадает с параметром `dim`, усреднение векторов предметов образа дает покомпонентное среднее, а пустой список векторов возвращает `None`, как заложено в вызывающем коде.

Модуль `outfit`: на синтетических векторах проверяется порядок кандидатов по косинусу (убывание, первый не слабее следующих). Оценка совместимости оборачивается в условный тест: если в тестовом каталоге есть веса, пара предметов разных ролей должна получить оценку в диапазоне от 0 до 1 (поле `score` в ответе). Сборка образов тестируется с подменой внутренней функции оценки на константы, чтобы отладить перебор и ранжирование без загрузки реальных файлов весов.

Тесты API обращаются к развернутым маршрутам: `/api/health`, редирект с корня на `/docs`, `/attributes` с файлом и без, JSON на `outfit/score`. Также проверяется, что приходят ошибки при неверной схеме тела.

Общие фикстуры поднимают тестовый HTTP-клиент `FastAPI` и небольшие растровые данные для `multipart`. Настройки прогона задаются в конфигурации `pytest`. Безусловных тестов больше двадцати – этого хватает, чтобы прикрыть основные ветки ML-сервиса.

Интеграция проверялась тем же `docker-compose`, что и для локальной разработки, без отдельного тестового кластера.

Для сквозных сценариев (добавление вещи, сборка образов, дозаполнение слота, поиск похожих комплектов) контролировался порядок вызовов ML, согласованность JSON и корректность отображения в интерфейсе.

Также ручной прогон на разных фото по типу, цвету и качеству подтвердил работоспособность цепочки.

Пример тестирования автоматического определения атрибутов через клиентскую часть приведен на рисунке 25, а пример составления образов из реального гардероба пользователя моделью совместимости приведен на рисунке 26.

Далее производились замеры на пользовательском компьютере среднего класса. `remove-bg` на кадре до 2048 px по длинной стороне укладывается примерно в одну секунду – хорошо для шага добавления вещи. Классификация атрибутов – около 100 мс на GPU и около 500 мс на CPU. Эмбединг того же кадра – тот же порядок, что и атрибуты, из-за общего дорогого прохода сверточной части. Вызов `outfit/score` укладывается в десятки миллисекунд на запрос – заметно быстрее глубоких голов, поэтому оценку совместимости можно многократно запрашивать в одном сценарии.

Более того, сверка HTTP-ответа атрибутов с прямым прогоном тех же весов в среде обучения дала полное совпадение предсказаний на одинаковых файлах: перенос весов, предобработка и код применения модели не меняют выход модели. При совпадении итоговых меток численные оценки из отчета по валидации и тесту можно трактовать как ориентир и для ответов по REST (Representational State Transfer), без отдельной калибровки под сетевой вызов.

Добавление вещи

Проверьте фото и добавьте вещь в гардероб



Название

Чёрная рубашка

Категория

Рубашка

Цвет

Чёрный

Сезон

Лето

Стиль

Повседневный

Добавить в гардероб

Выбрать другое фото

Отменить

+

Рисунок 25 – Проверка автоматического определения атрибутов через клиентскую часть. Источник: Составлено автором

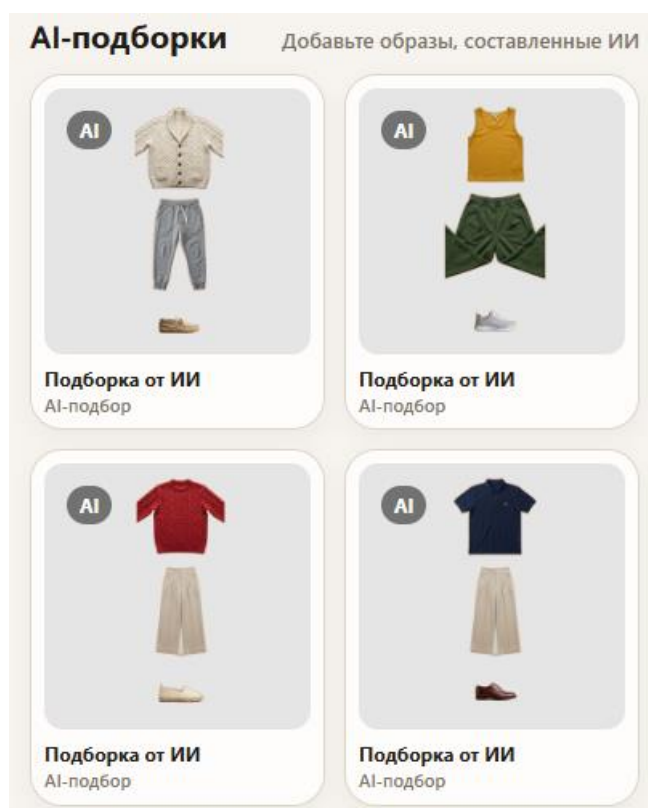


Рисунок 26 – Пример рекомендованных образов на основе реального гардероба в клиентской части. Источник: Составлено автором

Уровни модульных, интеграционных и пользовательских тестов закрыты. Pytest-набор пригоден для регрессии при правках. Замеры подтверждают приемлемое время отклика в интерактивных сценариях при типичных объемах данных и частоте запросов.

4.5 Выводы по четвертой главе

Четвертая глава сводит обученные модели к работающему контуру: обозначена обобщенная архитектура приложения, логика ML-сервиса на FastAPI, сценарии его вызова и результаты проверок на стенде.

ML-сервис держит разделение точки входа, слоя маршрутов, прикладной логики и конфигурации. Наружу отдаются маршруты удаления фона, атрибутов, эмбединга, оценки и сборки образа, рекомендаций по слоту и поиска похожих комплектов, а также служебные пути вроде health. Схемы тел запросов задают типизированные модели и контракт OpenAPI. Веса подгружаются лениво и кэшируются в памяти процесса. Веса и метки живут в

отдельном томе. Загрузка восстанавливает архитектуру по сохраненным ключам и совмещается с предобучением ImageNet. В работе задействован параллельный прогон голов на GPU или CPU. Предобработка изображения совпадает с валидацией при обучении.

Типовые цепочки выглядят следующим образом: при добавлении вещи последовательно вызываются `remove-bg`, атрибуты и эмбединг, затем результат сохраняется в хранилищах приложения. Сборка, дозаполнение и поиск похожих опираются на уже сохраненные векторы без повторного прогона сверточного извлекателя на каждое действие.

Модульные тесты на `pytest`, интеграция в `docker-compose`, ручной прогон сценариев и замеры задержек подкрепляют уверенность в корректности сборки. Совпадение предсказаний по HTTP и в среде обучения подтверждает корректность переноса моделей.

Связка подготовки данных, обучения моделей и выноса их в отдельный сервис согласуется с общей архитектурой приложения. Код и тесты дают опору для расширения функций и сопровождения без полной переписи ядра.

ЗАКЛЮЧЕНИЕ

Работа раскрывает проект «PlechiKi» для упрощения учета личного гардероба и подбора образов из уже заведенных вещей. Описаны и реализованы классификация атрибутов по фото, оценка совместимости комплекта и ранжирование кандидатных наборов, а вычислительный блок вынесен в отдельный ML-сервис с HTTP-доступом для основного приложения.

Главной целью работы являлось повышение релевантности рекомендаций по подбору одежды посредством разработки и интеграции модуля машинного обучения.

Удалось достичь следующего:

1) Проведен анализ предметной области и существующих цифровых сервисов управления гардеробом, выявлены ключевые ограничения в части автоматизации атрибутов и оценки совместимости.

2) Собраны и подготовлены наборы данных: Kaggle Fashion Product Images для классификаторов атрибутов и Polyvore Outfits для модели совместимости.

3) Выполнена предобработка данных – очистка и канонизация меток, сегментация фона, формирование обучающей, валидационной и тестовой выборок со стратификацией.

4) Проведено экспериментальное исследование одиннадцати архитектур для многоклассовой классификации атрибутов одежды, по результатам отобраны ConvNeXt-Tiny, ViT-B/16 и ResNet-50.

5) Проведено экспериментальное исследование классических и ансамблевых методов для оценки совместимости элементов гардероба на эмбедингах атрибутивных моделей.

6) Выполнен сравнительный анализ алгоритмов по метрикам F1, accuracy и ROC-AUC, выбраны оптимальные модели для каждого атрибута и для оценки совместимости.

7) Разработан переборно-ранжирующий алгоритм формирования рекомендаций образов из пользовательского гардероба на основе оценки совместимости.

8) Разработанный ML-модуль интегрирован в веб-приложение «Plechiki» в виде отдельного FastAPI-сервиса и проверен на реальных данных пользовательского гардероба.

По итогам сравнения одиннадцати архитектур для каждого из четырех атрибутов отобран лучший backbone: ConvNeXt-Tiny для типа предмета (macro-F1 = 0,899 на тесте) и назначения (0,891), ViT-B/16 для цвета (0,715), ResNet-50 для сезона (0,617). Оценку совместимости выполняет ансамблевый VotingClassifier на эмбедингах атрибутивных моделей: accuracy = 0,861, ROC-AUC = 0,928 на тестовой части Polyvore при среднем времени вывода около 3 мс на образ. Атрибутивные классификаторы отрабатывают за 100-500 мс в зависимости от наличия GPU, что укладывается в интерактивный сценарий добавления вещи.

Разработанный сервис сокращает ручной ввод при добавлении вещи – четыре атрибута определяются автоматически по фотографии – и ранжирует образы из имеющегося гардероба без перебора вручную. Модуль развернут в составе рабочего прототипа «Plechiki» и проверен на реальных данных пользовательского гардероба.

Дальнейшая работа может идти по прикладной и по методической линии. Имеет смысл расширить атрибуты (стиль, материал, узор), категории (аксессуары, головные уборы, сумки) и правила сборки образа (учет явных предпочтений пользователя). Со стороны методов – более новые или мультимодальные сети, самообучение при нехватке разметки, активное обучение для пополнения обучающей выборки по мере эксплуатации. Эти шаги поднимут качество и гибкость системы под запросы пользователя.

Резюмируя, поставленная цель достигнута, а задачи решены. Модуль ускоряет перевод личного гардероба в цифровой формат, упрощает учет вещей и помогает подбирать образы на каждый день.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. A New Textiles Economy: Redesigning Fashion's Future // Ellen MacArthur Foundation. URL: <https://ellenmacarthurfoundation.org/a-new-textiles-economy> (дата обращения: 10.04.2026).
2. Cherry K. Why You Only Wear 20% of Your Wardrobe // Forbes. URL: <https://www.forbes.com/sites/kateashford/2018/02/23/wardrobe-unused> (дата обращения: 10.04.2026).
3. Baumeister R. F., Tierney J. Willpower: Rediscovering the Greatest Human Strength. — New York: Penguin Press, 2011. — 291 p.
4. Государственный доклад «О состоянии и об охране окружающей среды Российской Федерации в 2023 году» // Министерство природных ресурсов и экологии Российской Федерации. URL: https://www.mnr.gov.ru/docs/gosudarstvennye_doklady/ (дата обращения: 12.04.2026).
5. Stylebook: Closet and Wardrobe App // Stylebook. URL: <https://www.stylebookapp.com> (дата обращения: 15.04.2026).
6. Whering: Digital Wardrobe and Outfit Planner // Whering. URL: <https://whering.co.uk> (дата обращения: 15.04.2026).
7. Acloset: AI Fashion Assistant // Acloset. URL: <https://acloset.app> (дата обращения: 15.04.2026).
8. Goodfellow I., Bengio Y., Courville A. Deep Learning. — Cambridge: MIT Press, 2016. — 800 p.
9. Zou Z., Chen K., Shi Z., Guo Y., Ye J. Object Detection in 20 Years: A Survey // Proceedings of the IEEE. — 2023. — Vol. 111, № 3. — P. 257-276.
10. Kirillov A., Mintun E., Ravi N., Mao H., Rolland C., Gustafson L., Xiao T., Whitehead S., Berg A. C., Lo W.-Y., Dollár P., Girshick R. Segment Anything // Proceedings of the IEEE/CVF International Conference on Computer Vision. — Paris, 2023. — P. 4015-4026.

11. Zheng S., Yang F., Kiapour M. H., Piramuthu R. ModaNet: A Large-Scale Street Fashion Dataset with Polygon Annotations // Proceedings of the 26th ACM International Conference on Multimedia. — Seoul, 2018. — P. 1670-1678.
12. Liu Z., Luo P., Qiu S., Wang X., Tang X. DeepFashion: Powering Robust Clothes Recognition and Retrieval with Rich Annotations // Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. — Las Vegas, 2016. — P. 1096-1104.
13. Musgrave K., Belongie S., Lim S.-N. A Metric Learning Reality Check // Proceedings of the European Conference on Computer Vision. — Glasgow, 2020. — P. 681-699.
14. Hastie T., Tibshirani R., Friedman J. The Elements of Statistical Learning: Data Mining, Inference, and Prediction. — 2nd ed. — New York: Springer, 2009. — 745 p.
15. Krizhevsky A., Sutskever I., Hinton G. E. ImageNet Classification with Deep Convolutional Neural Networks // Advances in Neural Information Processing Systems. — Lake Tahoe, 2012. — Vol. 25. — P. 1097-1105.
16. He K., Zhang X., Ren S., Sun J. Deep Residual Learning for Image Recognition // Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. — Las Vegas, 2016. — P. 770-778.
17. Tan M., Le Q. V. EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks // Proceedings of the 36th International Conference on Machine Learning. — Long Beach, 2019. — P. 6105-6114.
18. Howard A. G., Zhu M., Chen B. et al. MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications // arXiv. URL: <https://arxiv.org/abs/1704.04861> (дата обращения: 16.04.2026).
19. Liu Z., Mao H., Wu C.-Y., Feichtenhofer C., Darrell T., Xie S. A ConvNet for the 2020s // Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. — New Orleans, 2022. — P. 11976-11986.

20. Woo S., Park S., Lee J.-Y., Kweon I. S. ConvNeXt V2: Co-designing and Scaling ConvNets with Masked Autoencoders // Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. — Vancouver, 2023. — P. 16133-16142.
21. Dosovitskiy A., Beyer L., Kolesnikov A. et al. An Image Is Worth 16×16 Words: Transformers for Image Recognition at Scale // International Conference on Learning Representations. — Vienna, 2021. — 22 p.
22. Touvron H., Cord M., Douze M., Massa F., Sablayrolles A., Jégou H. Training Data-Efficient Image Transformers and Distillation through Attention // Proceedings of the 38th International Conference on Machine Learning. — Online, 2021. — P. 10347-10357.
23. Liu Z., Lin Y., Cao Y., Hu H., Wei Y., Zhang Z., Lin S., Guo B. Swin Transformer: Hierarchical Vision Transformer using Shifted Windows // Proceedings of the IEEE/CVF International Conference on Computer Vision. — Montreal, 2021. — P. 10012-10022.
24. Liu Z., Hu H., Lin Y., Wei Z., Zhang Z., Cao H., Hu Z., Wang H., Wei F. Swin Transformer V2: Scaling Up Capacity and Resolution // Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. — New Orleans, 2022. — P. 12009-12019.
25. Ridnik T., Ben-Baruch E., Noy A., Zelnik-Manor L. ImageNet-21K Pretraining for the Masses // Advances in Neural Information Processing Systems. Datasets and Benchmarks Track. — Online, 2021. — 13 p.
26. Zhuang F., Qi Z., Duan K., Xi D., Zhu Y., Zhu H., Xiong H., He Q. A Comprehensive Study of Transfer Learning // Proceedings of the IEEE. — 2021. — Vol. 109, № 1. — P. 43-76.
27. Chen X., Xie S., He K. An Empirical Study of Training Self-Supervised Vision Transformers // Proceedings of the IEEE/CVF International Conference on Computer Vision. — Montreal, 2021. — P. 9640-9649.
28. He K., Chen X., Xie S., Li Y., Dollár P., Girshick R. Masked Autoencoders Are Scalable Vision Learners // Proceedings of the IEEE/CVF Conference on

- Computer Vision and Pattern Recognition. — New Orleans, 2022. — P. 16000-16009.
29. Bao H., Dong L., Piao S., Wei F. BEiT: BERT Pre-Training of Image Transformers // International Conference on Learning Representations. — Online, 2022. — 17 p.
30. Caron M., Touvron H., Misra I., Jégou H., Mairal J., Bojanowski P., Joulin A. Emerging Properties in Self-Supervised Vision Transformers // Proceedings of the IEEE/CVF International Conference on Computer Vision. — Montreal, 2021. — P. 9650-9660.
31. Oquab M., Darcet T., Moutakanni T., Amir H., Khalidov M., Fernandez P., Joulin F., Mairal J., Jégou H. DINOv2: Learning Robust Visual Features without Supervision // Transactions on Machine Learning Research. URL: <https://arxiv.org/abs/2304.07193> (дата обращения: 22.04.2026).
32. Radford A., Kim J. W., Hallacy C. et al. Learning Transferable Visual Models from Natural Language Supervision // Proceedings of the 38th International Conference on Machine Learning. — Online, 2021. — P. 8748-8763.
33. Veit A., Kovacs B., Bell S. et al. Learning Visual Clothing Style with Heterogeneous Dyadic Co-occurrences // Proceedings of the IEEE International Conference on Computer Vision. — Santiago, 2015. — P. 4642-4650.
34. Chen W., Huang P., Xu J. et al. POG: Personalized Outfit Generation for Fashion Recommendation at Alibaba iFashion // Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. — Anchorage, 2019. — P. 2662-2670.
35. Vasileva M. I., Plummer B. A., Dusad K. et al. Learning Type-Aware Embeddings for Fashion Compatibility // Proceedings of the European Conference on Computer Vision. — Munich, 2018. — P. 390-405.
36. Han X., Wu Z., Jiang Y.-G., Davis L. S. Learning Fashion Compatibility with Bidirectional LSTMs // Proceedings of the 25th ACM International Conference on Multimedia. — Mountain View, 2017. — P. 1078-1086.

- 37.Cui Z., Li Z., Wu S., Zhang X.-Y., Wang L. Dressing as a Whole: Outfit Compatibility Learning Based on Node-wise Graph Neural Networks // The World Wide Web Conference. — San Francisco, 2019. — P. 307-317.
- 38.Bommasani R., Hudson D. A., Adeli E. et al. On the Opportunities and Risks of Foundation Models // arXiv. URL: <https://arxiv.org/abs/2108.07258> (дата обращения: 15.04.2026).
- 39.Sokolova M., Lapalme G. A Systematic Analysis of Performance Measures for Classification Tasks // Information Processing & Management. — 2009. — Vol. 45, № 4. — P. 427-437.
- 40.Tsoumakas G., Katakis I., Vlahavas I. Mining Multi-Label Data // Data Mining and Knowledge Discovery Handbook. — Boston: Springer, 2010. — P. 667-685.
- 41.Järvelin K., Kekäläinen J. Cumulated Gain-Based Evaluation of IR Techniques // ACM Transactions on Information Systems. — 2002. — Vol. 20, № 4. — P. 422-446.
- 42.Han X., Wu Z., Huang P. X. et al. Fashion Compatibility Learning via End-to-End Network // Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. — Salt Lake City, 2018. — P. 5022-5031.
- 43.Van Rossum G., Drake F. L. Python 3 Reference Manual. — Scotts Valley: CreateSpace, 2009. — 242 p.
- 44.Paszke A., Gross S., Massa F. et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library // Advances in Neural Information Processing Systems. — Vancouver, 2019. — Vol. 32. — P. 8024-8035.
- 45.Wightman R. PyTorch Image Models (timm) // GitHub. URL: <https://github.com/huggingface/pytorch-image-models> (дата обращения: 17.04.2026).
- 46.Buslaev A., Iglovikov V. I., Khvedchenya E. et al. Albumentations: Fast and Flexible Image Augmentations // Information. — 2020. — Vol. 11, № 2. — P. 125.

47. Müller S. G., Hutter F. TrivialAugment: Tuning-free Yet State-of-the-Art Data Augmentation // Proceedings of the IEEE/CVF International Conference on Computer Vision. — Montreal, 2021. — P. 774-782.
48. Pedregosa F., Varoquaux G., Gramfort A. et al. Scikit-learn: Machine Learning in Python // Journal of Machine Learning Research. — 2011. — Vol. 12. — P. 2825-2830.
49. Biewald L. Experiment Tracking with Weights and Biases // Weights & Biases. URL: <https://www.wandb.com> (дата обращения: 18.04.2026).
50. Ramírez S. FastAPI: Modern, Fast Web Framework for Building APIs with Python // FastAPI. URL: <https://fastapi.tiangolo.com> (дата обращения: 18.04.2026).
51. Bai J., Lu F., Zhang K. et al. ONNX: Open Neural Network Exchange // GitHub. URL: <https://github.com/onnx/onnx> (дата обращения: 18.04.2026).
52. Paleyes A., Urma R.-G., Lawrence N. D. Challenges in Deploying Machine Learning: A Survey of Case Studies // ACM Computing Surveys. — 2022. — Vol. 55, № 6. — Art. 114. — 29 p.
53. Kreuzberger D., Kühl N., Hirschl S. Machine Learning Operations (MLOps): Overview, Definition, and Architecture // IEEE Access. — 2023. — Vol. 11. — P. 31866-31879.
54. Fashion Product Images Dataset // Kaggle. URL: <https://www.kaggle.com/datasets/paramaggarwal/fashion-product-images-dataset> (дата обращения: 20.04.2026).
55. DeepFashion: Category and Attribute Prediction Benchmark // DeepFashion. URL: <https://mmlab.ie.cuhk.edu.hk/projects/DeepFashion.html> (дата обращения: 20.04.2026).
56. Polyvore Outfits Dataset // GitHub. URL: <https://github.com/mvasil/fashion-compatibility> (дата обращения: 20.04.2026).
57. Qin X., Zhang Z., Huang C., Dehghan M., Zaiane O. R., Jagersand M. U2-Net: Going Deeper with Nested U-Structure for Salient Object Detection // Pattern Recognition. — 2020. — Vol. 106. — Art. 107404. — 12 p.

- 58.Ridnik T., Sharir G., Ben-Cohen A., Ben-Baruch E., Noy A. ML-Decoder: Scalable and Versatile Classification Head // Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision. — Waikoloa, 2023. — P. 32-41.
- 59.Prokhorenkova L., Gusev G., Vorobev A., Dorogush A. V., Gulin A. CatBoost: Unbiased Boosting with Categorical Features // Advances in Neural Information Processing Systems. — Montreal, 2018. — Vol. 31. — P. 6638-6648.
- 60.Loshchilov I., Hutter F. Decoupled Weight Decay Regularization // Proceedings of the International Conference on Learning Representations. — New Orleans, 2019. — 18 p.
- 61.Swagger UI // Swagger.io. URL: <https://swagger.io/tools/swagger-ui/> (дата обращения: 25.04.2026).

СПИСОК ИЛЛЮСТРАТИВНОГО МАТЕРИАЛА

1. Примеры изображений из датасета для классификации атрибутов одежды. Источник: Составлено автором на основе источников [54]
2. Примеры изображений из датасета для обучения модели совместимости. Источник: Составлено автором на основе источников [56]
3. Примеры результатов предобработки изображений одежды с использованием rembg и cloth UNet. Источник: Составлено автором на основе источников [54, 57]
4. Схема конвейера предобработки изображений одежды. Источник: Составлено автором
5. Гистограмма распределения предметов одежды по типу (наиболее представленные классы). Источник: Составлено автором на основе источников [54]
6. Гистограмма распределения предметов одежды по группам (верхняя одежда, нижняя одежда, обувь). Источник: Составлено автором на основе источников [54]
7. Гистограмма распределения предметов одежды по значениям атрибута цвета в составе сформированного набора данных. Источник: Составлено автором на основе источников [54]
8. Гистограмма распределения предметов одежды по значениям атрибута сезона в составе сформированного набора данных. Источник: Составлено автором на основе источников [54]
9. Гистограмма распределения предметов одежды по значениям атрибута назначения в составе сформированного набора данных. Источник: Составлено автором на основе источников [54]
10. Распределение типов одежды по сезонам в датасете. Источник: Составлено автором на основе источников [54]
11. Схема классификации атрибутов одежды. Источник: Составлено автором

- 12.Схема цепочки обработки оценки совместимости комплекта. Источник:
Составлено автором
- 13.Нормированная матрица несоответствий для головы классификации
типа предмета одежды на тестовой подвыборке. Источник: Составлено
автором
- 14.Нормированная матрица несоответствий для головы классификации
базового цвета на тестовой подвыборке. Источник: Составлено автором
- 15.Нормированная матрица несоответствий для головы классификации
сезонной принадлежности на тестовой подвыборке. Источник:
Составлено автором
- 16.Нормированная матрица несоответствий для головы классификации
назначения на тестовой подвыборке. Источник: Составлено автором
- 17.Примеры предсказаний модели классификации типа одежды на тестовой
выборке. Источник: Составлено автором
- 18.Пример предсказания обученной модели в задаче оценки совместимости
на тестовой выборке. Источник: Составлено автором
- 19.Блок-схема алгоритма формирования образов. Источник: Составлено
автором
- 20.Пример рекомендации недостающей вещи. Источник: Составлено
автором
- 21.Схема архитектуры разработанной системы. Источник: Составлено
автором
- 22.Схема слоев сервиса обработки запросов и их взаимодействия.
Источник: Составлено автором
- 23.Интерактивная документация REST API сервиса в Swagger UI.
Составлено автором на основе источников [61]
- 24.Диаграмма последовательности при добавлении предмета в гардероб.
Источник: Составлено автором
- 25.Проверка автоматического определения атрибутов через клиентскую
часть. Источник: Составлено автором

26.Пример рекомендованных образов на основе реального гардероба в клиентской части. Источник: Составлено автором